AWT Assignments-1 Code

Krishna Patil
PRN: 125M1H025
Subject: AWT

Task 1

Question 1

Problem Statement

Create a browser-based student management application that validates, adds, edits, removes, and displays student records.

Code

Filename: app.js

```
const students = [
  {
    id: "S001",
    name: "Aarav",
    marks: 78,
    subjects: ["Maths", "Science"]
  },
  {
    id: "S002",
    name: "Diya",
    marks: 92,
    subjects: ["Maths", "English"]
  },
  {
    id: "S003",
    name: "Kabir",
    marks: 35,
    subjects: ["Science", "English"]
  }
];

let editingId = null;

const $ = (selector) => document.querySelector(selector);

function showMessage(text, type = "error") {
  $("#message").innerHTML = `<div class="message ${type}">${text}</div>`;
}

function resetForm() {
  $("#studentForm").reset();
  $("#studentId").disabled = false;
  $("#formTitle").textContent = "Add Student";
  $("#submitButton").textContent = "Add Student";
  $("#cancelEdit").hidden = true;
  editingId = null;
}

function render() {
  const query = $("#search").value.trim().toLowerCase();

  const filtered = students
    .filter((student) =>
      student.id.toLowerCase().includes(query) ||
      student.name.toLowerCase().includes(query)
    )
    .sort((a, b) =>
      $("#sort").value === "desc"
        ? b.marks - a.marks
        : a.marks - b.marks
    );

  $("#rows").innerHTML = filtered.length
    ? filtered.map((student) => `
      <tr>
        <td>${student.id}</td>
        <td>${student.name}</td>
        <td>${student.subjects.join(", ")}</td>
```

```
        <td>${student.marks}</td>
        <td>
          <span class="badge ${student.marks > 40 ? "success-text" : "danger-text"}">
            ${student.marks > 40 ? "Passed" : "Failed"}
          </span>
        </td>
        <td>
          <div class="actions">
            <button onclick="editStudent('${student.id}')">Edit</button>
            <button class="danger" onclick="deleteStudent('${student.id}')">Delete</button>
          </div>
        </td>
      </tr>
    `).join("")
    : `<tr><td colspan="6" class="empty">No students found.</td></tr>`;

  $("#count").textContent = students.length;
  $("#passed").textContent = students.filter((student) => student.marks > 40).length;

  if (students.length) {
    const topper = students.reduce((highest, student) =>
      student.marks > highest.marks ? student : highest
    );
    $("#highest").textContent = `${topper.name} (${topper.marks})`;
  } else {
    $("#highest").textContent = "—";
  }
}

function editStudent(id) {
  const student = students.find((item) => item.id === id);

  if (!student) {
    showMessage("Student record not found.");
    return;
  }

  editingId = id;
  $("#formTitle").textContent = `Edit Student — ${id}`;
  $("#studentId").value = student.id;
  $("#studentId").disabled = true;
  $("#studentName").value = student.name;
  $("#studentMarks").value = student.marks;
  $("#studentSubjects").value = student.subjects.join(", ");
  $("#submitButton").textContent = "Update Student";
  $("#cancelEdit").hidden = false;
  $("#message").innerHTML = "";
}

function deleteStudent(id) {
  const index = students.findIndex((student) => student.id === id);

  if (index === -1) {
    showMessage("Student record not found.");
    return;
  }

  students.splice(index, 1);

  if (editingId === id) {
    resetForm();
  }

  showMessage("Student deleted successfully.", "success");
  render();
}

$("#studentForm").addEventListener("submit", (event) => {
  event.preventDefault();

  const id = $("#studentId").value.trim().toUpperCase();
  const name = $("#studentName").value.trim();
  const marks = Number($("#studentMarks").value);
  const subjects = $("#studentSubjects").value
    .split(",")
    .map((subject) => subject.trim())
    .filter(Boolean);

  if (!id || !name || !subjects.length || marks < 0 || marks > 100) {
    showMessage("Please enter valid student details.");
    return;
  }

  if (editingId) {
    const student = students.find((item) => item.id === editingId);

    if (!student) {
      showMessage("Student record no longer exists.");
      return;
    }

    student.name = name;
    student.marks = marks;
    student.subjects = subjects;
```

```
    showMessage("Student record updated successfully.", "success");
    resetForm();
    render();
    return;
  }

  if (students.some((student) => student.id === id)) {
    showMessage(`Duplicate student ID "${id}". Record was not added.`);
    return;
  }

  students.push({ id, name, marks, subjects });
  showMessage("Student added successfully.", "success");
  resetForm();
  render();
});

$("#cancelEdit").addEventListener("click", () => {
  resetForm();
  $("#message").innerHTML = "";
});

$("#search").addEventListener("input",  render);
$("#sort").addEventListener("change", render);

render();
```

Filename: index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Management System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<main class="container">
  <section class="hero">
    <h1>Student Management System</h1>
    <p>Add, edit, search, sort and analyze student records.</p>
  </section>

  <section class="card">
    <h2 id="formTitle">Add Student</h2>
    <form id="studentForm">
      <input id="studentId" placeholder="Student ID (e.g. S001)" required>
      <input id="studentName" placeholder="Student name" required>
      <input id="studentMarks" type="number" min="0" max="100" placeholder="Marks" required>
      <input id="studentSubjects" placeholder="Subjects, comma separated" required>
      <button id="submitButton">Add Student</button>
      <button id="cancelEdit" type="button" class="secondary" hidden>Cancel Edit</button>
    </form>
    <div id="message"></div>
  </section>

  <section class="grid">
    <div class="card stat"><b>Total Students</b><strong id="count">0</strong></div>
    <div class="card stat"><b>Highest Marks</b><strong id="highest">—</strong></div>
    <div class="card stat"><b>Passed</b><strong id="passed">0</strong></div>
  </section>

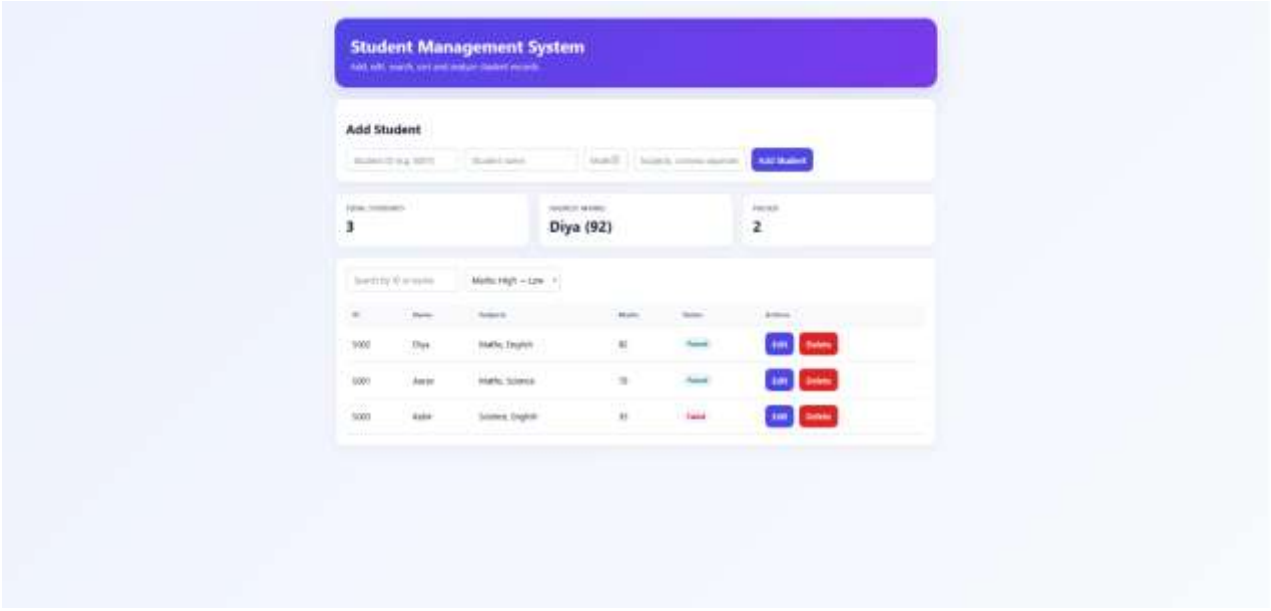
  <section class="card">
    <div class="controls">
      <input id="search" placeholder="Search by ID or name">
      <select id="sort">
        <option value="desc">Marks: High → Low</option>
        <option value="asc">Marks: Low → High</option>
      </select>
    </div>
    <br>
    <table>
      <thead>
        <tr>
          <th>ID</th><th>Name</th><th>Subjects</th><th>Marks</th>
          <th>Status</th><th>Actions</th>
        </tr>
      </thead>
      <tbody id="rows"></tbody>
    </table>
  </section>
</main>
<script src="app.js"></script>
</body>
</html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
```

```
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
```

```
  border: 1px solid var(--border);
  border-radius:  12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```



Task 2

Question 1

Problem Statement

Create a browser-based library management application for maintaining books, categories, issue and return status, and book actions.

Code

Filename: app.js

```
const books = [
  {
    id: "B001",
    title: "Clean Code",
    author: "Robert C. Martin",
    category: "Programming",
    publisher: "Prentice Hall",
    price: 650,
    available: true
  },
  {
    id: "B002",
    title: "Atomic Habits",
    author: "James Clear",
    category: "Self Help",
    publisher: "Avery",
    price: 499,
    available: true
  },
  {
    id: "B003",
    title: "The Alchemist",
    author: "Paulo Coelho",
    category: "Fiction",
    publisher: "HarperOne",
    price: 350,
    available: false
  },
  {
    id: "B004",
    title: "JavaScript: The Good Parts",
    author: "Douglas Crockford",
    category: "Programming",
    publisher: "O'Reilly",
    price:  799,
    available: true
  }
];

let editingId = null;
const $ = (selector) => document.querySelector(selector);

function message(text, type = "error") {
  $("#message").innerHTML = `<div class="message ${type}">${text}</div>`;
}

function populateCategories() {
  const current = $("#categoryFilter").value;
  const categories = [...new Set(books.map((book) => book.category))]
    .sort((a, b) => a.localeCompare(b));

  $("#categoryFilter").innerHTML =
    `<option value="">All Categories</option>` +
    categories.map((category) => `<option value="${category}">${category}</option>`).join("");

  if (categories.includes(current)) {
    $("#categoryFilter").value = current;
  }
}

function resetForm() {
  $("#bookForm").reset();
  $("#bookId").disabled = false;
  $("#formTitle").textContent = "Add Book";
  $("#submitButton").textContent = "Add Book";
  $("#cancelEdit").hidden = true;
  editingId = null;
}

function render() {
  populateCategories();

  const query = $("#search").value.trim().toLowerCase();
  const category = $("#categoryFilter").value;
```

```
   const filtered = books
     .filter((book) => {
       const searchable = `${book.title} ${book.author} ${book.publisher}`.toLowerCase();
       return searchable.includes(query) && (!category || book.category === category);
     })
     .sort((a, b) =>
       $("#sort").value === "title"
         ? a.title.localeCompare(b.title)
         : a.price - b.price
     );

   $("#rows").innerHTML = filtered.length
     ? filtered.map((book) => `
       <tr>
         <td>${book.id}</td>
         <td>${book.title}</td>
         <td>${book.author}</td>
         <td>${book.category}</td>
         <td>₹ ${book.price.toLocaleString()}</td>
         <td><span class="badge">${book.available ? "Available" : "Issued"}</span></td>
         <td>
           <div class="actions">
             <button onclick="editBook('${book.id}')">Edit</button>
             <button class="warning" onclick="toggleAvailability('${book.id}')">
               ${book.available ? "Issue" : "Return"}
             </button>
             <button class="danger" onclick="deleteBook('${book.id}')">Delete</button>
           </div>
         </td>
       </tr>
     `).join("")
     : `<tr><td colspan="7" class="empty">No books found.</td></tr>`;

   $("#total").textContent = books.length;
   $("#availableCount").textContent = books.filter((book) => book.available).length;

   if (books.length) {
     const cheapest = Math.min(...books.map((book) => book.price));
     const costliest = Math.max(...books.map((book) => book.price));
     $("#range").textContent = `₹ ${cheapest} / ₹ ${costliest}`;
   } else {
     $("#range").textContent = "—";
   }
 }

 function editBook(id) {
   const book = books.find((item) => item.id === id);

   if (!book) {
     message("Book record not found.");
     return;
   }

   editingId = id;
   $("#formTitle").textContent = `Edit Book — ${id}`;
   $("#bookId").value = book.id;
   $("#bookId").disabled = true;
   $("#title").value = book.title;
   $("#author").value = book.author;
   $("#category").value = book.category;
   $("#publisher").value = book.publisher;
   $("#price").value = book.price;
   $("#available").value = String(book.available);
   $("#submitButton").textContent = "Update Book";
   $("#cancelEdit").hidden = false;
   $("#message").innerHTML = "";
 }

 function deleteBook(id) {
   const index = books.findIndex((book) => book.id === id);

   if (index === -1) {
     message("Book record not found.");
     return;
   }

   books.splice(index, 1);

   if (editingId === id) {
     resetForm();
   }

   message("Book deleted successfully.", "success");
   render();
 }

 function toggleAvailability(id) {
   const book = books.find((item) => item.id === id);

   if (!book) {
     message("Book record not found.");
     return;
   }
```

```
  book.available = !book.available;
  message(`Book marked as ${book.available ? "available" : "issued"}.`, "success");
  render();
}

$("#bookForm").addEventListener("submit", (event) => {
  event.preventDefault();

  const id = $("#bookId").value.trim().toUpperCase();
  const bookData = {
    id,
    title: $("#title").value.trim(),
    author: $("#author").value.trim(),
    category: $("#category").value.trim(),
    publisher: $("#publisher").value.trim(),
    price: Number($("#price").value),
    available: $("#available").value === "true"
  };

  if (
    !id ||
    !bookData.title ||
    !bookData.author ||
    !bookData.category ||
    !bookData.publisher ||
    bookData.price < 0
  ) {
    message("Please enter valid book details.");
    return;
  }

  if (editingId) {
    const book = books.find((item) => item.id === editingId);

    if (!book) {
      message("Book record no longer exists.");
      return;
    }

    Object.assign(book, {
      title: bookData.title,
      author: bookData.author,
      category: bookData.category,
      publisher: bookData.publisher,
      price: bookData.price,
      available: bookData.available
    });

    resetForm();
    message("Book record updated successfully.", "success");
    render();
    return;
  }

  if (books.some((book) => book.id === id)) {
    message(`Duplicate Book ID "${id}". Record was not added.`);
    return;
  }

  books.push(bookData);
  resetForm();
  message("Book added successfully.", "success");
  render();
});

$("#cancelEdit").addEventListener("click", () => {
  resetForm();
  $("#message").innerHTML = "";
});

$("#search").addEventListener("input",  render);
$("#categoryFilter").addEventListener("change",  render);
$("#sort").addEventListener("change", render);

render();
```

Filename: index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Library Management System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<main class="container">
  <section class="hero"><h1>Library Management System</h1><p>Maintain complete book records and availability.</p></
section>

  <section class="card">
    <h2 id="formTitle">Add Book</h2>
    <form id="bookForm">
```

```
      <input id="bookId" placeholder="Book ID (e.g. B001)" required>
      <input id="title" placeholder="Title" required>
      <input id="author" placeholder="Author" required>
      <input id="category" placeholder="Category" required>
      <input id="publisher" placeholder="Publisher" required>
      <input id="price" type="number" min="0" placeholder="Price" required>
      <select id="available"><option value="true">Available</option><option value="false">Issued</option></select>
      <button id="submitButton">Add Book</button>
      <button id="cancelEdit" type="button" class="secondary" hidden>Cancel Edit</button>
    </form>
    <div id="message"></div>
  </section>

  <section class="grid">
    <div class="card stat"><b>Total Books</b><strong id="total">0</strong></div>
    <div class="card stat"><b>Available</b><strong id="availableCount">0</strong></div>
    <div class="card stat"><b>Cheapest / Costliest</b><strong id="range">—</strong></div>
  </section>

  <section class="card">
    <div class="controls">
      <input id="search" placeholder="Search title, author or publisher">
      <select id="categoryFilter"><option value="">All Categories</option></select>
      <select id="sort">
        <option value="title">Title A–Z</option>
        <option value="price">Price Low–High</option>
      </select>
    </div>
    <br>
    <table>
      <thead><tr><th>ID</th><th>Title</th><th>Author</th><th>Category</th><th>Price</th><th>Status</th><th>Actions</
th></tr></thead>
      <tbody id="rows"></tbody>
    </table>
  </section>
</main>
<script src="app.js"></script>
</body>
</html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius:  16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
```

```
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```


Library Management System
Add Book
4
3
₹350 / ₹799

Task 3

Question 1

Problem Statement

Create an employee management application that demonstrates employee data handling with validation, editing, deletion, and class getter/setter behavior.

Code

Filename: app.js

```
class Employee {
    constructor(id, name, designation, salary) {
        this.id = id;
        this.name = name;
        this.designation = designation;

        this._salary = 0;
        this.salary = salary;
    }

    get salary() {
        return this._salary;
    }

    set salary(value) {
        const numericSalary = Number(value);

        if (!Number.isFinite(numericSalary) || numericSalary < 0) {
            throw new Error("Salary must be a valid non-negative number.");
        }

        this._salary = numericSalary;
    }

    getAnnualSalary() {
        return this.salary * 12;
    }
}

const employees = [
  new Employee("E001", "Aarav", "Developer", 45000),
  new Employee("E002", "Diya", "Designer", 62000),
  new Employee("E003", "Meera", "Manager", 85000)
];

let editingId = null;
const $ = (selector) => document.querySelector(selector);

function showMessage(text, type = "error") {
  $("#message").innerHTML = `<div class="message ${type}">${text}</div>`;
}

function resetForm() {
   $("#form").reset();
  $("#formTitle").textContent = "Add Employee";
  $("#form button:first-of-type").textContent = "Add Employee";
  $("#cancel").hidden = true;
  editingId = null;
}

function render() {
    const displayEmployees = employees.map((employee) => ({
        id: employee.id,
        name: employee.name,
        designation: employee.designation,
        salary: employee.salary,
        adjustedSalary: employee.salary < 50000 ? employee.salary * 1.10 : employee.salary
    }));

    $("#rows").innerHTML = displayEmployees.map((employee) => `
        <tr>
            <td>${employee.id}</td>
            <td>${employee.name}</td>
            <td>${employee.designation}</td>
            <td>₹ ${employee.salary.toFixed(0)}</td>
            <td>₹ ${employee.adjustedSalary.toFixed(0)}</td>
            <td>
                <div class="actions">
                    <button onclick="editEmployee('${employee.id}')">Edit</button>
                    <button class="danger" onclick="deleteEmployee('${employee.id}')">Delete</button>
                </div>
            </td>
        </tr>
    `).join("");
```

```
    $("#count").textContent = employees.length;

    $("#payroll").textContent =
        `₹${employees
            .reduce((sum, employee) => sum + employee.salary, 0)
            .toFixed(0)}`;

    $("#high").textContent =
        employees.filter((employee) => employee.salary > 60000).length;
}

function editEmployee(id) {
  const employee = employees.find((item) => item.id === id);

  if (!employee) {
    showMessage("Employee record not found.");
    return;
  }

  editingId = id;
  $("#formTitle").textContent = `Edit Employee — ${id}`;
  $("#name").value = employee.name;
  $("#designation").value = employee.designation;
  $("#salary").value = employee.salary;
  $("#form button:first-of-type").textContent = "Update Employee";
  $("#cancel").hidden = false;
}

function deleteEmployee(id) {
  const index = employees.findIndex((employee) => employee.id === id);

  if (index === -1) {
    showMessage("Employee record not found.");
    return;
  }

  employees.splice(index, 1);

  if (editingId === id) {
    resetForm();
  }

  showMessage("Employee deleted successfully.", "success");
  render();
}

$("#form").addEventListener("submit", (event) => {
  event.preventDefault();

  const name = form.name.value.trim();
  const designation = form.designation.value.trim();
  const salary = Number(form.salary.value);
  if (!name || !designation || !Number.isFinite(salary) || salary < 0) {
    showMessage("Please enter valid employee details.");
    return;
  }

  try {
    if (editingId) {
      const employee = employees.find((item) => item.id === editingId);

      if (!employee) {
        showMessage("Employee record no longer exists.");
        return;
      }

      employee.name = name;
      employee.designation = designation;
      employee.salary = salary;

      resetForm();
      showMessage("Employee updated successfully.", "success");
      render();
      return;
    }

    const id = `E${String(Date.now()).slice(-4)}`;

    if (employees.some((employee) => employee.id === id)) {
      showMessage("Duplicate employee ID. Please try again.");
      return;
    }

    employees.push(new Employee(id, name, designation, salary));

    // map() is used to create the salary-adjusted employee view.
    employees.map((employee) => {
      if (employee.salary < 50000 && employee.id === id) {
        employee.salary *= 1.10;
      }
      return employee;
    });

    resetForm();
```

```
    showMessage("Employee added successfully.", "success");
    render();
  } catch (error) {
    showMessage(error.message);
   }
});

$("#cancel").addEventListener("click", () => {
  resetForm();
  $("#message").innerHTML = "";
});

render();
```

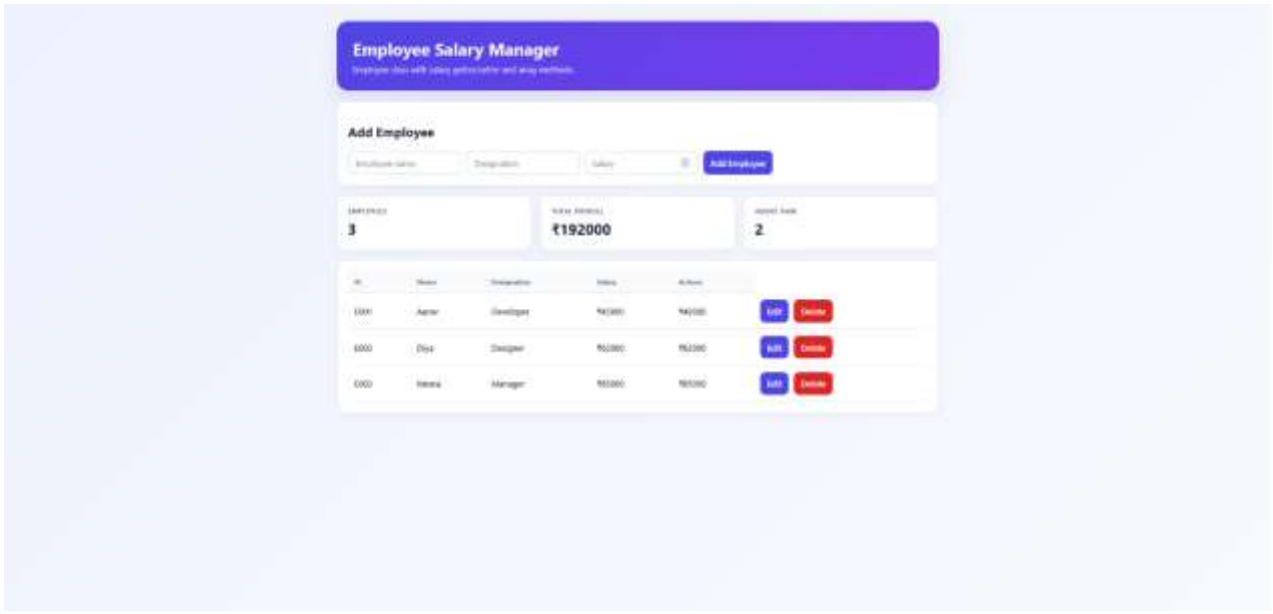
Filename: index.html

```
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1.0"><title>Employee Salary
Manager</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container">
<section class="hero"><h1>Employee Salary Manager</h1><p>Employee class with salary getter/setter and array methods.</
p></section>
<section class="card"><h2 id="formTitle">Add Employee</h2><form id="form"><input id="name" placeholder="Employee name"
required><input id="designation" placeholder="Designation" required><input id="salary" type="number" min="0"
placeholder="Salary" required><button>Add Employee</button><button id="cancel" type="button" class="secondary"
hidden>Cancel Edit</button></form><div id="message"></div></section>
<section class="grid"><div class="card stat"><b>Employees</b><strong id="count">0</strong></div><div class="card
stat"><b>Total Payroll</b><strong id="payroll">₹ 0</strong></div><div class="card stat"><b>Above ₹ 60K</b><strong
id="high">0</strong></div></section>
<section class="card"><table><thead><tr><th>ID</th><th>Name</th><th>Designation</th><th>Salary</th><th>Actions</th></
tr></thead><tbody id="rows"></tbody></table></section>
</main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
```

```
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```


Employee Salary Manager
Add Employee
3
₹192000
2

Task 4

Question 1

Problem Statement

Create a hospital records application for maintaining patient records with validation, editing, deletion, discharge, and reporting actions.

Code

Filename: app.js

```
class Patient {
  constructor(id, name, age, disease, doctor, bill, status = "Admitted") {
    this.id = id;
    this.name = name;
    this.age = age;
    this.disease = disease;
    this.doctor = doctor;
    this.billAmount = bill;
    this.status = status;
  }

  get bill() {
    return this.billAmount;
  }

  set bill(value) {
    if (!Number.isFinite(value) || value < 0) {
      throw new Error("Bill amount must be non-negative.");
    }

    this.billAmount = value;
  }

  discharge() {
    this.status = "Discharged";
  }
}

class InPatient extends Patient {
  constructor(...args) {
    super(...args);
    this.type = "In-Patient";
  }
}

const patients = [
  new InPatient("P001", "Aarav Sharma", 32, "Fever", "Dr. Rao", 18000),
  new InPatient("P002", "Diya Patil", 45, "Fracture", "Dr. Mehta", 75000),
  new InPatient("P003", "Kabir Joshi", 28, "Migraine", "Dr. Shah", 9000, "Discharged")
];

let editingId = null;
const $ = (selector) => document.querySelector(selector);

function message(text, type = "error", target = "#message") {
  $(target).innerHTML = `<div class="message ${type}">${text}</div>`;
}

function resetForm() {
   $("#form").reset();
  $("#formTitle").textContent = "Add Patient";
  $("#form button:first-of-type").textContent = "Add Patient";
  $("#cancel").hidden = true;
  editingId = null;
}

function render(list = patients) {
  $("#rows").innerHTML = list.length
    ? list.map((patient) => {
      const { id, name, age, disease, doctor, bill, status } = patient;

      return `
        <tr>
          <td>${id}</td>
          <td>${name}</td>
          <td>${age}</td>
          <td>${disease}</td>
          <td>${doctor}</td>
          <td>₹ ${bill.toLocaleString()}</td>
          <td><span class="badge">${status}</span></td>
          <td>
            <div class="actions">
              <button onclick="editPatient('${id}')">Edit</button>
              <button onclick="dischargePatient('${id}')">Discharge</button>
              <button class="danger" onclick="deletePatient('${id}')">Delete</button>
```

```
            </div>
          </td>
        </tr>
      `;
    }).join("")
    : `<tr><td colspan="8" class="empty">No patient records found.</td></tr>`;

  $("#admitted").textContent = patients.filter((patient) => patient.status === "Admitted").length;
  $("#discharged").textContent = patients.filter((patient) => patient.status === "Discharged").length;
  $("#revenue").textContent = `₹ ${patients.reduce((sum, patient) => sum + patient.bill, 0).toLocaleString()}`;
}

function editPatient(id) {
  const patient = patients.find((item) => item.id === id);

  if (!patient) {
    message("Patient record not found.");
    return;
  }

  editingId = id;
  $("#formTitle").textContent = `Edit Patient — ${id}`;
  $("#name").value = patient.name;
  $("#age").value = patient.age;
  $("#disease").value = patient.disease;
  $("#doctor").value = patient.doctor;
  $("#bill").value = patient.bill;
  $("#status").value = patient.status;
  $("#form button:first-of-type").textContent = "Update Patient";
  $("#cancel").hidden = false;
}

function dischargePatient(id) {
  const patient = patients.find((item) => item.id === id);

  if (!patient) {
    message("Patient record not found.");
    return;
  }

  patient.discharge();
  message("Patient marked as discharged.", "success");
  render();
}

function deletePatient(id) {
  const index = patients.findIndex((patient) => patient.id === id);

  if (index === -1) {
    message("Patient record not found.");
    return;
  }

  patients.splice(index, 1);

  if (editingId === id) {
    resetForm();
  }

  message("Patient deleted successfully.", "success");
  render();
}

function filterBills() {
  const threshold = Number($("#threshold").value);

  if (!Number.isFinite(threshold) || threshold < 0) {
    message("Enter a valid bill threshold.", "error", "#reportMessage");
    return;
  }

  const filtered = patients.filter((patient) => patient.bill > threshold);
  message(
    `${filtered.length} patient(s) have bills above ₹ ${threshold.toLocaleString()}.`,
    "success",
    "#reportMessage"
  );
  render(filtered);
}

$("#form").addEventListener("submit", (event) => {
  event.preventDefault();

  const data = {
    name: $("#name").value.trim(),
    age: Number($("#age").value),
    disease: $("#disease").value.trim(),
    doctor: $("#doctor").value.trim(),
    bill: Number($("#bill").value),
    status: $("#status").value
  };

  if (!data.name || data.age < 0 || !data.disease || !data.doctor || data.bill < 0) {
    message("Please enter valid patient details.");
```

```
    return;
  }

  try {
    if (editingId) {
      const patient = patients.find((item) => item.id === editingId);

      if (!patient) {
        message("Patient record no longer exists.");
        return;
      }

      patient.name = data.name;
      patient.age = data.age;
      patient.disease = data.disease;
      patient.doctor = data.doctor;
      patient.bill = data.bill;
      patient.status = data.status;

      resetForm();
      message("Patient record updated successfully.", "success");
      render();
      return;
    }

    const id = `P${String(Date.now()).slice(-4)}`;

    if (patients.some((patient) => patient.id === id)) {
      message("Duplicate patient ID. Record was not added.");
      return;
    }

    patients.push(
      new InPatient(
        id,
        data.name,
        data.age,
        data.disease,
        data.doctor,
        data.bill,
        data.status
      )
    );

    resetForm();
    message("Patient added successfully.", "success");
    render();
  } catch (error) {
    message(error.message);
  }
});

$("#cancel").addEventListener("click", () => {
  resetForm();
  $("#message").innerHTML = "";
});

render();
```

Filename: index.html

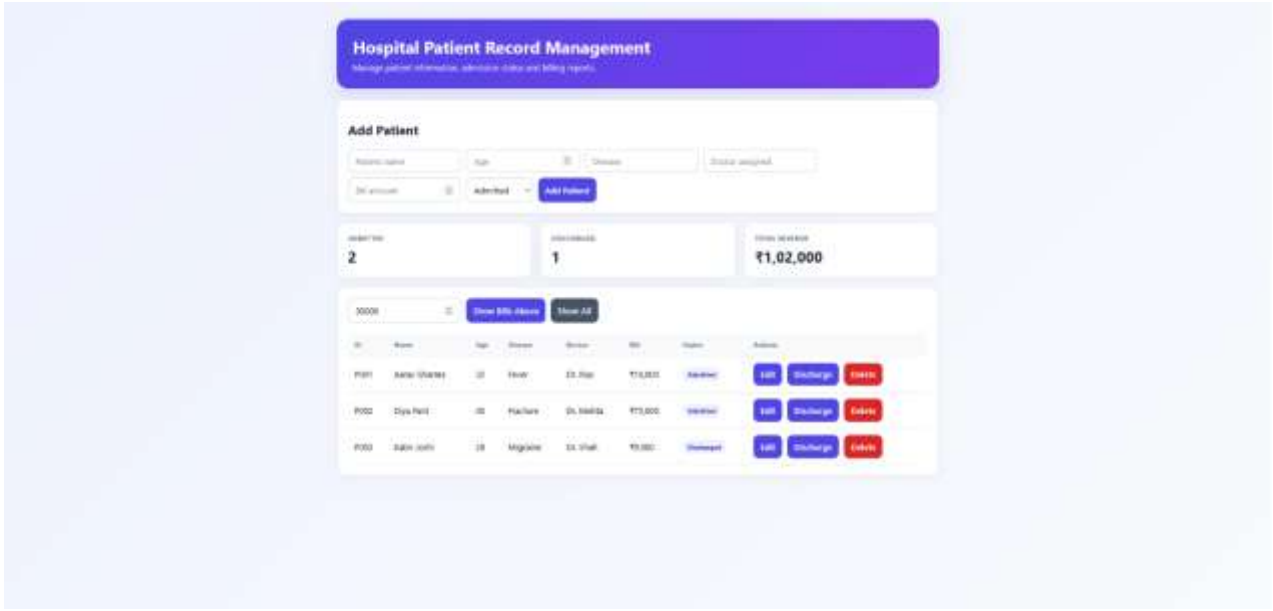
```
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1.0"><title>Hospital
Patient Records</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container">
<section class="hero"><h1>Hospital Patient Record Management</h1><p>Manage patient information, admission status and
billing reports.</p></section>
<section class="card"><h2 id="formTitle">Add Patient</h2><form id="form"><input id="name" placeholder="Patient name"
required><input id="age" type="number" min="0" placeholder="Age" required><input id="disease" placeholder="Disease"
required><input id="doctor" placeholder="Doctor assigned" required><input id="bill" type="number" min="0"
placeholder="Bill amount" required><select id="status"><option>Admitted</option><option>Discharged</option></
select><button>Add Patient</button><button id="cancel" type="button" class="secondary" hidden>Cancel Edit</button></
form><div id="message"></div></section>
<section class="grid"><div class="card stat"><b>Admitted</b><strong id="admitted">0</strong></div><div class="card
stat"><b>Discharged</b><strong id="discharged">0</strong></div><div class="card stat"><b>Total Revenue</b><strong
id="revenue">₹ 0</strong></div></section>
<section class="card"><div class="controls"><input id="threshold" type="number" value="50000" min="0" placeholder="Bill
threshold"><button onclick="filterBills()">Show Bills Above</button><button class="secondary" onclick="render()">Show
All</button></div><div id="reportMessage"></div><br><table><thead><tr><th>ID</th><th>Name</th><th>Age</th><th>Disease</
th><th>Doctor</th><th>Bill</th><th>Status</th><th>Actions</th></tr></thead><tbody id="rows"></tbody></table></section>
</main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
```

```
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
```

```
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```



Task 5

Question 1

Problem Statement

Create an employee attendance and salary application that manages employee records, attendance, payroll calculations, salary slips, and reports.

Code

Filename: app.js

```
const employees = [
  {
    id: "E001",
    name: "Aarav",
    basicSalary: 50000,
    present: 24,
    workingDays: 26,
    overtimeHours: 8,
    deductions: 1500
  },
  {
    id: "E002",
    name: "Diya",
    basicSalary: 60000,
    present: 26,
    workingDays: 26,
    overtimeHours: 4,
    deductions: 2000
  }
];

const OVERTIME_RATE = 350;
let editingId = null;
const $ = (selector) => document.querySelector(selector);

function calculateSalary(employee) {
  const attendancePay =
    employee.basicSalary *
    (employee.present / employee.workingDays);

  const overtimePay = employee.overtimeHours * OVERTIME_RATE;
  const gross = attendancePay + overtimePay;
  const net = gross - employee.deductions;

  return { attendancePay, overtimePay, gross, net };
}

function showMessage(text, type = "error") {
  $("#message").innerHTML = `<div class="message ${type}">${text}</div>`;
}

function resetForm() {
  $("#form").reset();
  $("#days").value = 26;
  $("#overtime").value = 0;
  $("#deduction").value = 0;
  $("#formTitle").textContent = "Add Employee";
  $("#form button:first-of-type").textContent = "Add Employee";
  $("#cancel").hidden = true;
  editingId = null;
}

function render() {
  const payroll = employees.map((employee) => ({
    ...employee,
    ...calculateSalary(employee)
  }));

  $("#rows").innerHTML = payroll.length
    ? payroll.map((employee) => `
      <tr>
        <td>${employee.id}</td>
        <td>${employee.name}</td>
        <td>${employee.present}/${employee.workingDays}</td>
        <td>${employee.overtimeHours}h</td>
        <td>₹ ${employee.gross.toFixed(0)}</td>
        <td>₹ ${employee.deductions.toFixed(0)}</td>
        <td><b>₹ ${employee.net.toFixed(0)}</b></td>
        <td>
          <div class="actions">
            <button onclick="editEmployee('${employee.id}')">Edit</button>
            <button onclick="viewSlip('${employee.id}')">Salary Slip</button>
            <button class="danger" onclick="deleteEmployee('${employee.id}')">Delete</button>
          </div>
        </td>
```

```
      </tr>
    `).join("")
    : `<tr><td colspan="8" class="empty">No payroll records.</td></tr>`;

  $("#count").textContent = employees.length;
  $("#payroll").textContent =
    `₹${payroll.reduce((sum, employee) => sum + employee.net, 0).toFixed(0)}`;

  if (employees.length) {
    const best = employees.reduce((a, b) =>
      b.present / b.workingDays > a.present / a.workingDays ? b : a
    );
    $("#best").textContent = best.name;
  } else {
    $("#best").textContent = "—";
  }
}

function editEmployee(id) {
  const employee = employees.find((item) => item.id === id);

  if (!employee) {
    showMessage("Employee record not found.");
    return;
  }

  editingId = id;
  $("#formTitle").textContent = `Edit Employee — ${id}`;
  $("#name").value = employee.name;
  $("#salary").value = employee.basicSalary;
  $("#present").value = employee.present;
  $("#days").value = employee.workingDays;
  $("#overtime").value = employee.overtimeHours;
  $("#deduction").value = employee.deductions;
  $("#form button:first-of-type").textContent = "Update Employee";
  $("#cancel").hidden = false;
}

function deleteEmployee(id) {
  const index = employees.findIndex((employee) => employee.id === id);

  if (index === -1) {
    showMessage("Employee record not found.");
    return;
  }

  employees.splice(index, 1);

  if (editingId === id) {
    resetForm();
  }

  showMessage("Employee deleted successfully.", "success");
  render();
}

function viewSlip(id) {
  const employee = employees.find((item) => item.id === id);

  if (!employee) {
    showMessage("Employee record not found.");
    return;
  }

  const salary = calculateSalary(employee);

   $("#slip").innerHTML = `
     <section class="card">
      <h2>Salary Slip — ${employee.name}</h2>
      <p><b>Employee ID:</b> ${employee.id}</p>
      <p><b>Attendance:</b> ${employee.present}/${employee.workingDays}</p>
      <p><b>Attendance Pay:</b> ₹${salary.attendancePay.toFixed(0)}</p>
      <p><b>Overtime Pay:</b> ₹${salary.overtimePay.toFixed(0)}</p>
      <p><b>Gross Salary:</b> ₹${salary.gross.toFixed(0)}</p>
      <p><b>Deductions:</b> ₹${employee.deductions.toFixed(0)}</p>
      <h2>Net Salary: ₹${salary.net.toFixed(0)}</h2>
    </section>
  `;
}

$("#form").addEventListener("submit", (event) => {
  event.preventDefault();

  const data = {
    name: $("#name").value.trim(),
    basicSalary: Number($("#salary").value),
    present: Number($("#present").value),
    workingDays: Number($("#days").value),
    overtimeHours: Number($("#overtime").value),
    deductions: Number($("#deduction").value)
  };

  if (
    !data.name ||
```

```
    data.basicSalary < 0 ||
    data.present < 0 ||
    data.workingDays <= 0 ||
    data.present > data.workingDays ||
    data.overtimeHours < 0 ||
    data.deductions < 0
  ) {
    showMessage("Please enter valid attendance and salary details.");
    return;
  }

  if (editingId) {
    const employee = employees.find((item) => item.id === editingId);

    if (!employee) {
      showMessage("Employee record no longer exists.");
      return;
    }

    Object.assign(employee, data);
    resetForm();
    showMessage("Employee payroll record updated successfully.", "success");
    render();
    return;
  }

  const id = `E${String(Date.now()).slice(-4)}`;

  if (employees.some((employee) => employee.id === id)) {
    showMessage("Duplicate employee ID. Record was not added.");
    return;
  }

  employees.push({ id, ...data });
  resetForm();
  showMessage("Employee added successfully.", "success");
  render();
});

$("#cancel").addEventListener("click", () => {
  resetForm();
  $("#message").innerHTML = "";
});

render();
```

Filename: index.html

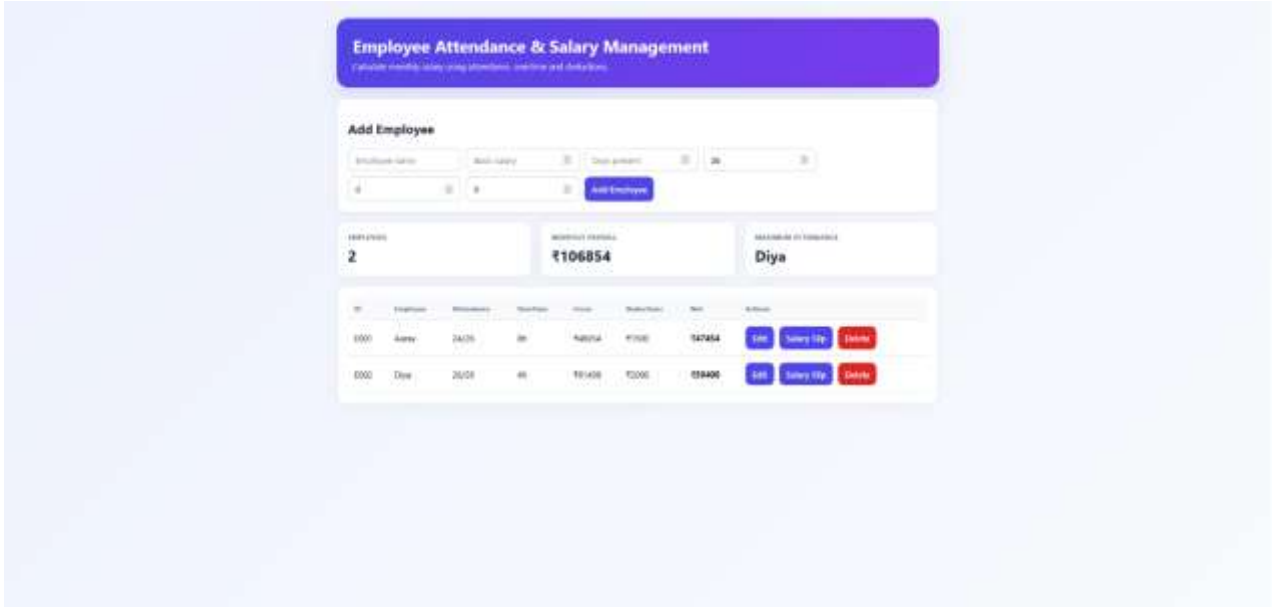
```
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1.0"><title>Attendance &
Payroll</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container">
<section class="hero"><h1>Employee Attendance & Salary Management</h1><p>Calculate monthly salary using attendance,
overtime and deductions.</p></section>
<section class="card"><h2 id="formTitle">Add Employee</h2><form id="form"><input id="name" placeholder="Employee name"
required><input id="salary" type="number" min="0" placeholder="Basic salary" required><input id="present" type="number"
min="0" placeholder="Days present" required><input id="days" type="number" min="1" placeholder="Working days"
value="26" required><input id="overtime" type="number" min="0" placeholder="Overtime hours" value="0" required><input
id="deduction" type="number" min="0" placeholder="Deductions" value="0"><button>Add Employee</button><button
id="cancel" type="button" class="secondary" hidden>Cancel Edit</button></form><div id="message"></div></section>
<section class="grid"><div class="card stat"><b>Employees</b><strong id="count">0</strong></div><div class="card
stat"><b>Monthly Payroll</b><strong id="payroll">₹ 0</strong></div><div class="card stat"><b>Maximum Attendance</
b><strong id="best">—</strong></div></section>
<section class="card"><table><thead><tr><th>ID</th><th>Employee</th><th>Attendance</th><th>Overtime</th><th>Gross</
th><th>Deductions</th><th>Net</th><th>Actions</th></tr></thead><tbody id="rows"></tbody></table></section>
<section id="slip"></section>
</main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
```

```
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
```

```
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```



Task 6

Question 1

Problem Statement

Create a task manager that allows users to add, complete, edit, remove, and persist tasks in browser storage.

Code

Filename: app.js

```
const storageKey = "tasks_final";
let tasks = JSON.parse(localStorage.getItem(storageKey) || "[]");
const $ = (selector) => document.querySelector(selector);

function save() {
  localStorage.setItem(storageKey,  JSON.stringify(tasks));
}

function render() {
  const filter = $("#filter").value;

  const visible = tasks.filter((task) =>
    filter === "all" ||
    (filter === "active" && !task.completed) ||
    (filter === "done" && task.completed)
  );

  $("#list").innerHTML = visible.length
    ? visible.map((task) => `
      <div class="item">
        <div   style="display:flex;justify-content:space-between;gap:10px;align-items:center">
          <div>
            <h3 style="${task.completed ? "text-decoration:line-through;opacity:.55" : ""}">
              ${task.text}
            </h3>
            <small class="muted">${task.completed ? "Completed" : "Pending"}</small>
          </div>
          <div class="actions">
             <button onclick="toggleTask('${task.id}')">
               ${task.completed ? "Undo" : "Complete"}
            </button>
            <button class="secondary" onclick="editTask('${task.id}')">Edit</button>
            <button class="danger" onclick="removeTask('${task.id}')">Remove</button>
          </div>
        </div>
      </div>
    `).join("")
    : '<div class="empty">No tasks found.</div>';
}

function toggleTask(id) {
  const task = tasks.find((item) => item.id === id);

  if (!task) return;

  task.completed = !task.completed;
  save();
  render();
}

function editTask(id) {
  const task = tasks.find((item) => item.id === id);

  if (!task) return;

  const updated = prompt("Edit task:", task.text);

  if (updated !== null && updated.trim()) {
    task.text = updated.trim();
    save();
    render();
  }
}

function removeTask(id) {
  tasks = tasks.filter((task) => task.id !== id);
  save();
  render();
}

function clearCompleted() {
  tasks = tasks.filter((task) => !task.completed);
  save();
  render();
}
```

```
$("#form").addEventListener("submit", (event) => {
  event.preventDefault();

  tasks.push({
    id: crypto.randomUUID(),
    text: $("#task").value.trim(),
    completed: false
  });

  event.target.reset();
  save();
  render();
});

$("#filter").addEventListener("change", render);

render();
```

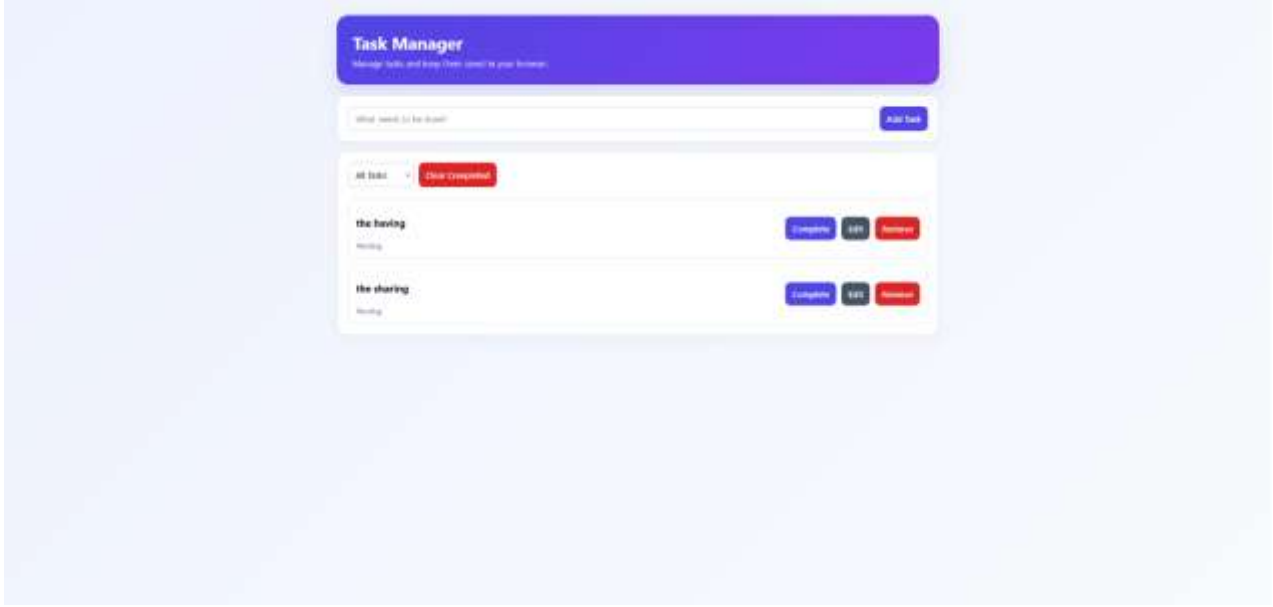
Filename: index.html

```
<!doctype html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-
scale=1.0"><title>Task Manager</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container"><section class="hero"><h1>Task Manager</h1><p>Manage tasks and keep them saved in your
browser.</p></section><section class="card"><form id="form"><input id="task" placeholder="What needs to be done?"
required style="flex:1"><button>Add Task</button></form></section><section class="card"><div class="controls"><select
id="filter"><option value="all">All Tasks</option><option value="active">Active</option><option value="done">Completed</
option></select><button class="danger" onclick="clearCompleted()">Clear Completed</button></div><br><div id="list"
class="list"></div></section></main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius:  16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius:  10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
```

```
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```



Task 7

Question 1

Problem Statement

Create a movie ticket booking application with movie and seat selection, ticket booking confirmation, cancellation, and persistent booking state.

Code

Filename: app.js

```
const movies = [
    {
        id: "M001",
        title: "Inception",
        genre: "Sci-Fi",
        price: 220,
        times: ["10:00 AM", "2:00 PM", "7:00 PM"]
    },
    {
        id: "M002",
        title: "Interstellar",
        genre: "Sci-Fi",
        price: 250,
        times: ["11:00 AM", "3:30 PM", "8:30 PM"]
    },
    {
        id: "M003",
        title: "The Dark Knight",
        genre: "Action",
        price: 200,
        times: ["12:00 PM", "5:00 PM", "9:00 PM"]
    }
];


// ---------------------------------------------------------------------------
// STORAGE
// ---------------------------------------------------------------------------

const storageKey = "movie_booking_corrected";

const bookedStorageKey = "movie_booked_seats_by_show";


// Default booked seats for different movie/show combinations.
// The key format is:
// movieId|showtime
//
// Example:
// M001|10:00 AM
// M001|2:00 PM
// M002|11:00 AM
// ---------------------------------------------------------------------------

const  defaultBookedSeats  =  {
    "M001|10:00 AM": ["A3", "B4", "D6"],
    "M001|2:00 PM": ["A1", "C5"],
    "M001|7:00 PM": ["B2", "E7"],

    "M002|11:00 AM": ["A4", "C3"],
    "M002|3:30 PM": ["B5", "D2"],
    "M002|8:30 PM": ["A6", "E4"],

    "M003|12:00 PM": ["C2", "D5"],
    "M003|5:00 PM": ["A2", "B6"],
    "M003|9:00 PM": ["C6", "E3"]
};


// ---------------------------------------------------------------------------
// LOAD BOOKED SEATS
// ---------------------------------------------------------------------------

let bookedSeatsByShow = JSON.parse(
    localStorage.getItem(bookedStorageKey) || "null"
);


// If no previous booking data exists,
// use the default booked seats.
if (!bookedSeatsByShow) {
    bookedSeatsByShow = { ...defaultBookedSeats };

    localStorage.setItem(
        bookedStorageKey,
```

```
        JSON.stringify(bookedSeatsByShow)
    );
}


// ------------------------------------------------------------------------
// LOAD CURRENT BOOKING
// ------------------------------------------------------------------------

let booking = JSON.parse(
    localStorage.getItem(storageKey) || "null"
) || {
    movieId: "M001",
    time: "10:00 AM",
    seats: [],
    confirmed: false
};


// ------------------------------------------------------------------------
// HELPER
// ------------------------------------------------------------------------

const $ = (selector) => document.querySelector(selector);


// ------------------------------------------------------------------------
// GET CURRENT MOVIE
// ------------------------------------------------------------------------

function currentMovie() {

    return movies.find(
        (movie) => movie.id === booking.movieId
    );
}


// ------------------------------------------------------------------------
// GET UNIQUE KEY FOR CURRENT MOVIE + SHOWTIME
// ------------------------------------------------------------------------

function getShowKey() {

    return `${booking.movieId}|${booking.time}`;
}


// ------------------------------------------------------------------------
// GET BOOKED SEATS FOR CURRENT SHOW
// ------------------------------------------------------------------------

function getBookedSeatsForCurrentShow() {

    const key = getShowKey();

    return bookedSeatsByShow[key] || [];
}


// ------------------------------------------------------------------------
// SAVE DATA
// ------------------------------------------------------------------------

function save() {

    localStorage.setItem(
        storageKey,
        JSON.stringify(booking)
    );

    localStorage.setItem(
        bookedStorageKey,
        JSON.stringify(bookedSeatsByShow)
    );
}


// ------------------------------------------------------------------------
// SHOW MESSAGE
// ------------------------------------------------------------------------

function showMessage(text, type = "success") {

    $("#message").innerHTML = `
        <div class="message ${type}">
            ${text}
        </div>
    `;
}


// ------------------------------------------------------------------------
// RENDER EVERYTHING
```

```
// ------------------------------------------------------------------------

function render() {

    const movie = currentMovie();

    if (!movie) {
        showMessage("Selected movie could not be found.", "error");
        return;
    }


    // ------------------------------------------------------------------------
    // MOVIES
    // ------------------------------------------------------------------------

    $("#movies").innerHTML = movies.map((item) => `

        <div class="movie">

            <h3>${item.title}</h3>

             <p class="muted">
                 ${item.genre}
            </p>

             <div class="price">
                   ₹${item.price}
            </div>

            <button
                onclick="selectMovie('${item.id}')"
            >
                ${
                    item.id === booking.movieId
                        ? "Selected"
                        : "Select Movie"
                }
            </button>

        </div>

    `).join("");


    // ------------------------------------------------------------------------
    // SELECTED MOVIE
    // ------------------------------------------------------------------------

    $("#selectedMovie").textContent = movie.title;


    // ------------------------------------------------------------------------
    // SHOW TIMES
    // ------------------------------------------------------------------------

    $("#times").innerHTML = movie.times.map((time) => `

        <button
            class="${
                time === booking.time
                    ? "secondary"
                    : ""
            }"

            onclick="selectTime('${time}')"
        >
            ${time}
         </button>

    `).join("");


    // ------------------------------------------------------------------------
    // BOOKED SEATS FOR CURRENT SHOW
    // ------------------------------------------------------------------------

    const bookedSeats = getBookedSeatsForCurrentShow();


    // ------------------------------------------------------------------------
    // SEAT GRID
    // ------------------------------------------------------------------------

    const allSeats = [
        "A",
        "B",
        "C",
        "D",
        "E"
    ].flatMap((row) =>

        Array.from(
            { length: 8 },
```

```
            (_, index) => `${row}${index + 1}`
        )

    );


    $("#seats").innerHTML = allSeats.map((seat) => {

        const isBooked =
            bookedSeats.includes(seat);

        const isSelected =
            booking.seats.includes(seat);


        return `

            <button
                 class="
                     seat
                    ${isBooked ? "booked" : ""}
                    ${isSelected ? "selected" : ""}
                "

                ${isBooked ? "disabled" : ""}

                onclick="toggleSeat('${seat}')"
            >
                ${seat}
            </button>

        `;

    }).join("");


    // ------------------------------------------------------------------------
    // BOOKING SUMMARY
    // ------------------------------------------------------------------------

    const total =
        booking.seats.length * movie.price;


    $("#summary").innerHTML = `

         <p>
             <b>Movie:</b>
             ${movie.title}
         </p>

         <p>
             <b>Show:</b>
             ${booking.time}
         </p>

         <p>
             <b>Seats:</b>
             ${
                 booking.seats.join(", ")
                 || "None"
             }
         </p>

         <p>
             <b>Tickets:</b>
             ${booking.seats.length}
         </p>

         <h2>
             Total: ₹ ${total}
         </h2>

         ${
             booking.confirmed
                 ? '<span class="badge">Ticket Booked</span>'
                 : ""
         }

    `;


    // ------------------------------------------------------------------------
    // BOOK BUTTON
    // ------------------------------------------------------------------------

    $("#bookButton").disabled =
        booking.seats.length === 0;
}


// ----------------------------------------------------------------------------
// SELECT MOVIE
// ----------------------------------------------------------------------------
```

```
function selectMovie(id) {

    const movie = movies.find(
        (item) => item.id === id
    );


    if (!movie) {

        showMessage(
            "Movie not found.",
            "error"
        );

        return;
    }


    // Change movie
    booking.movieId = id;


    // Automatically select the first
    // showtime of the new movie.
    booking.time = movie.times[0];


    // IMPORTANT:
    // Clear seats selected for the previous movie.
    booking.seats = [];


    // Booking confirmation is reset.
    booking.confirmed = false;


    save();

    render();

    showMessage(
        `Movie changed to ${movie.title}.`,
        "success"
    );
}


// ----------------------------------------------------------------------------
// SELECT SHOW TIME
// ----------------------------------------------------------------------------

function selectTime(time)  {

    const movie = currentMovie();


    if (!movie.times.includes(time)) {

        showMessage(
            "Invalid show timing.",
            "error"
        );

        return;
    }


    // Change showtime.
    booking.time = time;


    // IMPORTANT:
    // Clear the seats selected under
    // the previous showtime.
    booking.seats = [];


    // Reset confirmation.
    booking.confirmed = false;


    save();

    render();

    showMessage(
        `Show timing changed to ${time}.`,
        "success"
    );
}


// ----------------------------------------------------------------------------
// TOGGLE SEAT
```

```
// ---------------------------------------------------------------------------

function toggleSeat(seat) {

    const bookedSeats =
        getBookedSeatsForCurrentShow();


    // ---------------------------------------------------------------------------
    // DO NOT ALLOW BOOKED SEAT
    // ---------------------------------------------------------------------------

    if (bookedSeats.includes(seat)) {

        showMessage(
            `${seat} is already booked for this show.`,
            "error"
        );

        return;
    }


    // ---------------------------------------------------------------------------
    // SELECT / DESELECT
    // ---------------------------------------------------------------------------

    if (booking.seats.includes(seat)) {

        // Remove seat
        booking.seats =
            booking.seats.filter(
                (item) => item !== seat
            );

    } else {

        // Add seat
        booking.seats = [
            ...booking.seats,
            seat
        ];
    }


    // Selecting a seat means the current booking
    // is not considered confirmed anymore.
    booking.confirmed = false;


    save();

    render();
}


// ---------------------------------------------------------------------------
// BOOK TICKET
// ---------------------------------------------------------------------------

$("#bookButton").addEventListener(
    "click",
    () => {

        if (!booking.seats.length) {

            showMessage(
                "Please select at least one seat.",
                "error"
            );

            return;
        }


        const showKey = getShowKey();


        // ---------------------------------------------------------------------------
        // CHECK CURRENT SHOW'S BOOKED SEATS
        // ---------------------------------------------------------------------------

        if (!bookedSeatsByShow[showKey]) {

            bookedSeatsByShow[showKey] = [];
        }


        const alreadyBooked =
            bookedSeatsByShow[showKey];


        // ---------------------------------------------------------------------------
        // PREVENT DOUBLE BOOKING
```

```
        // ------------------------------------------------------------------

        const conflictingSeats =
            booking.seats.filter(
                (seat) =>
                    alreadyBooked.includes(seat)
            );


        if (conflictingSeats.length > 0) {

            showMessage(
                `The following seats are no longer available: ${conflictingSeats.join(", ")}`,
                "error"
            );


            // Remove conflicting seats
            booking.seats =
                booking.seats.filter(
                    (seat) =>
                        !conflictingSeats.includes(seat)
                );


            booking.confirmed = false;

            save();

            render();

            return;
        }


        // ------------------------------------------------------------------
        // ADD SEATS TO CURRENT SHOW
        // ------------------------------------------------------------------

        bookedSeatsByShow[showKey] = [
            ...new Set([
                ...alreadyBooked,
                ...booking.seats
            ])
        ];


        // Mark booking as confirmed.
        booking.confirmed = true;


        save();

        render();


        showMessage(
            `Ticket booked successfully for ${currentMovie().title} at ${booking.time}. Seats: ${booking.seats.join(",
")}.`,
            "success"
        );
    }
);


// ------------------------------------------------------------------------
// CANCEL CURRENT BOOKING SELECTION
// ------------------------------------------------------------------------

$("#cancelButton").addEventListener(
    "click",
    () => {

        booking.seats = [];

        booking.confirmed = false;

        save();

        render();

        showMessage(
            "Current booking selection cancelled.",
            "warning"
        );
    }
);


// ------------------------------------------------------------------------
// INITIAL RENDER
// ------------------------------------------------------------------------

render();
```

Filename: index.html

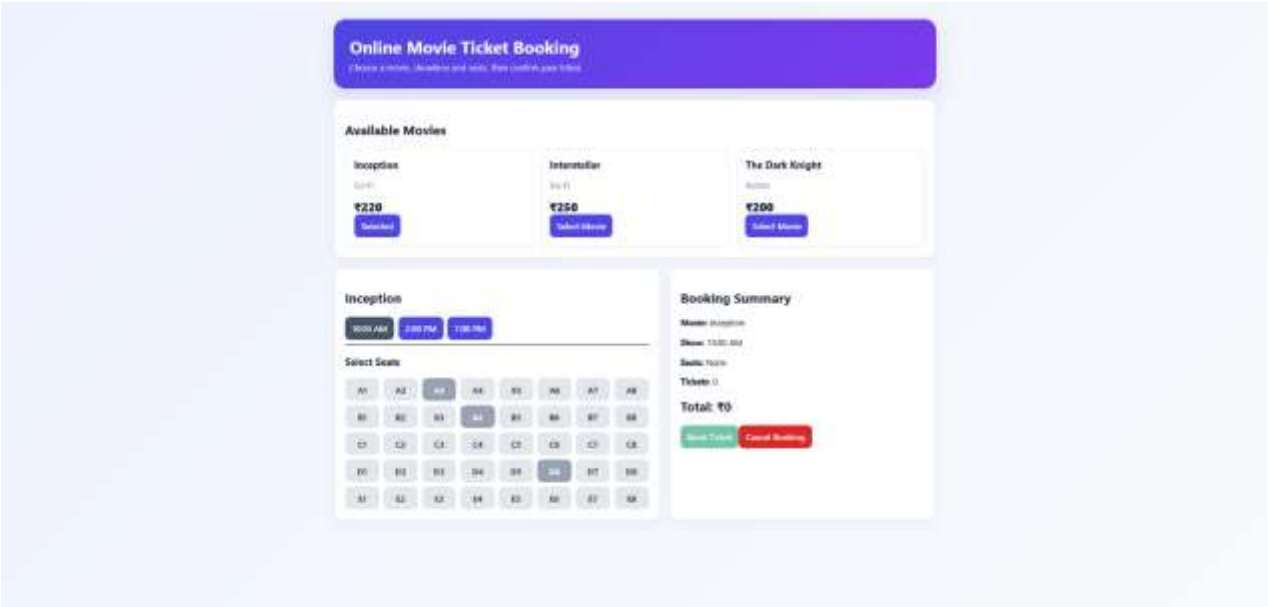
```
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1.0"><title>Movie Ticket
Booking</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container">
<section class="hero"><h1>Online Movie Ticket Booking</h1><p>Choose a movie, showtime and seats, then confirm your
ticket.</p></section>
<section class="card"><h2>Available Movies</h2><div id="movies" class="movie-grid"></div></section>
<section class="grid2">
<div class="card"><h2 id="selectedMovie">Choose a movie</h2><div id="times" class="pillbar"></div><hr><h3>Select Seats</
h3><div id="seats" class="seat-grid"></div></div>
<div class="card"><h2>Booking Summary</h2><div id="summary"></div><button id="bookButton" class="success">Book Ticket</
button><button id="cancelButton" class="danger">Cancel Booking</button><div id="message"></div></div>
</section>
</main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
```

```
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item  {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius:  12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}

.seat.booked { background:#98a2b3; color:#fff; cursor:not-allowed; opacity:1; text-decoration:line-through; }
.seat.selected { background:var(--primary); color:#fff; }
```



Task 8

Question 1

Problem Statement

Create an expense tracker that records transactions, supports editing, filtering, totals, and local storage persistence.

Code

Filename: app.js

```
const storageKey = "expense_tracker_corrected";
let transactions = JSON.parse(localStorage.getItem(storageKey) || "[]");
let editingId = null;

const $ = (selector) => document.querySelector(selector);

function save() {
  localStorage.setItem(storageKey,  JSON.stringify(transactions));
}

function showMessage(text, type = "success") {
  $("#message").innerHTML = `<div class="message ${type}">${text}</div>`;
}

function resetForm() {
   $("#form").reset();
  $("#date").valueAsDate = new Date();
  $("#formTitle").textContent = "Add Transaction";
  $("#submit").textContent = "Save Transaction";
  $("#cancel").hidden = true;
  editingId = null;
}

function getFilteredTransactions() {
  const query = $("#search").value.trim().toLowerCase();
  const category = $("#filterCat").value;
  const date = $("#filterDate").value;

  return transactions.filter((transaction) => {
    const matchesSearch =
      transaction.description.toLowerCase().includes(query);

    const matchesCategory =
      !category || transaction.category === category;

    const matchesDate =
      !date || transaction.date === date;

    return matchesSearch && matchesCategory && matchesDate;
  });
}

function render() {
  const filtered = getFilteredTransactions();

   $("#rows").innerHTML = filtered.length
    ? filtered.map((transaction) => `
      <tr>
        <td>${transaction.date}</td>
        <td>${transaction.type}</td>
        <td>${transaction.category}</td>
        <td>${transaction.description}</td>
        <td>₹ ${transaction.amount.toFixed(2)}</td>
        <td>
          <div class="actions">
            <button onclick="editTransaction('${transaction.id}')">Edit</button>
            <button class="danger" onclick="deleteTransaction('${transaction.id}')">Delete</button>
          </div>
        </td>
      </tr>
    `).join("")
    : `<tr><td colspan="6" class="empty">No transactions found.</td></tr>`;

  const income = transactions
    .filter((transaction) => transaction.type === "Income")
    .reduce((sum, transaction) => sum + transaction.amount, 0);

  const expenses = transactions
    .filter((transaction) => transaction.type === "Expense")
    .reduce((sum, transaction) => sum + transaction.amount, 0);

  $("#income").textContent = `₹ ${income.toFixed(2)}`;
  $("#expense").textContent = `₹ ${expenses.toFixed(2)}`;
  $("#balance").textContent = `₹ ${(income - expenses).toFixed(2)}`;
}
```

```
function editTransaction(id) {
  const transaction = transactions.find((item) => item.id === id);

  if (!transaction) {
    showMessage("Transaction not found.", "error");
    return;
  }

  editingId = id;

  $("#formTitle").textContent = "Edit Transaction";
  $("#date").value = transaction.date;
  $("#type").value = transaction.type;
  $("#category").value = transaction.category;
  $("#description").value = transaction.description;
  $("#amount").value = transaction.amount;
  $("#submit").textContent = "Update Transaction";
  $("#cancel").hidden = false;
  $("#message").innerHTML = "";
}

function deleteTransaction(id) {
  const exists = transactions.some((transaction) => transaction.id === id);

  if (!exists) {
    showMessage("Transaction not found.", "error");
    return;
  }

  transactions = transactions.filter((transaction) => transaction.id !== id);

  if (editingId === id) {
    resetForm();
  }

  save();
  showMessage("Transaction deleted successfully.");
  render();
}

$("#form").addEventListener("submit", (event) => {
  event.preventDefault();

  const transaction = {
    date: $("#date").value,
    type: $("#type").value,
    category: $("#category").value,
    description: $("#description").value.trim(),
    amount: Number($("#amount").value)
  };

  if (
    !transaction.date ||
    !transaction.description ||
    !Number.isFinite(transaction.amount) ||
    transaction.amount <= 0
  ) {
    showMessage("Please enter valid transaction details.", "error");
    return;
  }

  if (editingId) {
    const existing = transactions.find(
      (item) => item.id === editingId
    );

    if (!existing) {
      showMessage("Transaction no longer exists.", "error");
      return;
    }

    Object.assign(existing, transaction);
    resetForm();
    save();
    showMessage("Transaction updated successfully.");
    render();
    return;
  }

  transactions.push({
    id: crypto.randomUUID(),
    ...transaction
  });

  resetForm();
  save();
  showMessage("Transaction added successfully.");
  render();
});

$("#cancel").addEventListener("click", () => {
  resetForm();
  $("#message").innerHTML = "";
});
```

```
["search", "filterCat", "filterDate"].forEach((id) => {
  $(`#${id}`).addEventListener("input",  render);
});

$("#date").valueAsDate = new Date();
render();
```

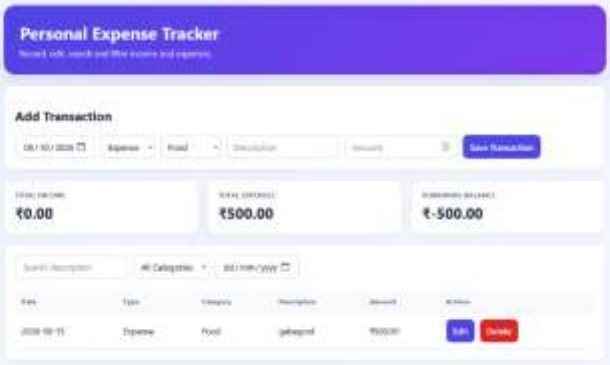
Filename: index.html

```
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1.0"><title>Expense
Tracker</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container">
<section class="hero"><h1>Personal Expense Tracker</h1><p>Record, edit, search and filter income and expenses.</p></
section>
<section class="card"><h2 id="formTitle">Add Transaction</h2><form id="form"><input id="date" type="date"
required><select id="type"><option>Expense</option><option>Income</option></select><select id="category"><option>Food</
option><option>Travel</option><option>Shopping</option><option>Salary</option><option>Bills</option><option>Other</
option></select><input id="description" placeholder="Description" required><input id="amount" type="number" min=".01"
step=".01" placeholder="Amount" required><button id="submit">Save Transaction</button><button id="cancel" type="button"
class="secondary" hidden>Cancel Edit</button></form><div id="message"></div></section>
<section class="grid"><div class="card stat"><b>Total Income</b><strong id="income">₹ 0</strong></div><div class="card
stat"><b>Total Expenses</b><strong id="expense">₹ 0</strong></div><div class="card stat"><b>Remaining Balance</b><strong
id="balance">₹ 0</strong></div></section>
<section class="card"><div class="controls"><input id="search" placeholder="Search description"><select
id="filterCat"><option value="">All Categories</option><option>Food</option><option>Travel</option><option>Shopping</
option><option>Salary</option><option>Bills</option><option>Other</option></select><input id="filterDate" type="date"></
div><br><table><thead><tr><th>Date</th><th>Type</th><th>Category</th><th>Description</th><th>Amount</th><th>Actions</
th></tr></thead><tbody id="rows"></tbody></table></section>
</main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius:  16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius:  10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
```

```
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```


Personal Expense Tracker
Add Transaction
₹0.00
₹500.00
₹-500.00

Task 9

Question 1

Problem Statement

Create a weather application that retrieves weather information for a city through an API and reports validation, network, and HTTP errors.

Code

Filename: app.js

```
const API_KEY = "ecd3dfc32a5448f653fa8da437a7df7d";
const API_URL = "https://api.openweathermap.org/data/2.5/weather";

const $ = (selector) => document.querySelector(selector);

function showResult(text, type = "message") {
  $("#result").className = `message ${type}`;
  $("#result").textContent = text;
}

async function getWeather(city) {
  if (!API_KEY || API_KEY === "ecd3dfc32a5448f653fa8da437a7df7d") {
    throw new Error(
      "OpenWeather API key is missing. Add your key to API_KEY in app.js."
    );
  }

  const url =
    `${API_URL}?q=${encodeURIComponent(city)}` +
    `&appid=${encodeURIComponent(API_KEY)}&units=metric`;

  let response;

  try {
    response = await fetch(url);
  } catch (error) {
    throw new Error(
      "Unable to connect to OpenWeather. Check your internet connection, " +
      "API key configuration, or browser/network restrictions."
    );
  }

  let data = null;

  try {
    data = await response.json();
  } catch {
    throw new Error("Weather service returned an invalid response.");
  }

  if (!response.ok) {
    if (response.status === 401) {
      throw new Error("OpenWeather API key is invalid or not activated yet.");
    }

    if (response.status === 404) {
      throw new Error(`City "${city}" was not found.`);
    }

    if (response.status === 429) {
      throw new Error("OpenWeather API request limit has been reached.");
    }

    throw new Error(
      data.message
        ? `OpenWeather error: ${data.message}.`
        : `Weather service failed with HTTP ${response.status}.`
    );
  }

  return data;
}

$("#form").addEventListener("submit", async (event) => {
  event.preventDefault();

  const city = $("#city").value.trim();

  if (!city) {
    showResult("Please enter a city name.", "error");
    return;
  }

  showResult("Loading weather data...", "warning");
```

```
  try {
    const data = await getWeather(city);

    $("#result").className = "card";
    $("#result").innerHTML = `
      <h2>${data.name}, ${data.sys.country}</h2>
      <div class="grid">
        <div>
          <span class="muted">Temperature</span>
          <h2>${data.main.temp} °C</h2>
        </div>
        <div>
          <span class="muted">Condition</span>
          <h2>${data.weather[0].description}</h2>
        </div>
        <div>
          <span class="muted">Humidity</span>
          <h2>${data.main.humidity}%</h2>
        </div>
      </div>
    `;
  } catch (error) {
    showResult(error.message, "error");
   }
});
```

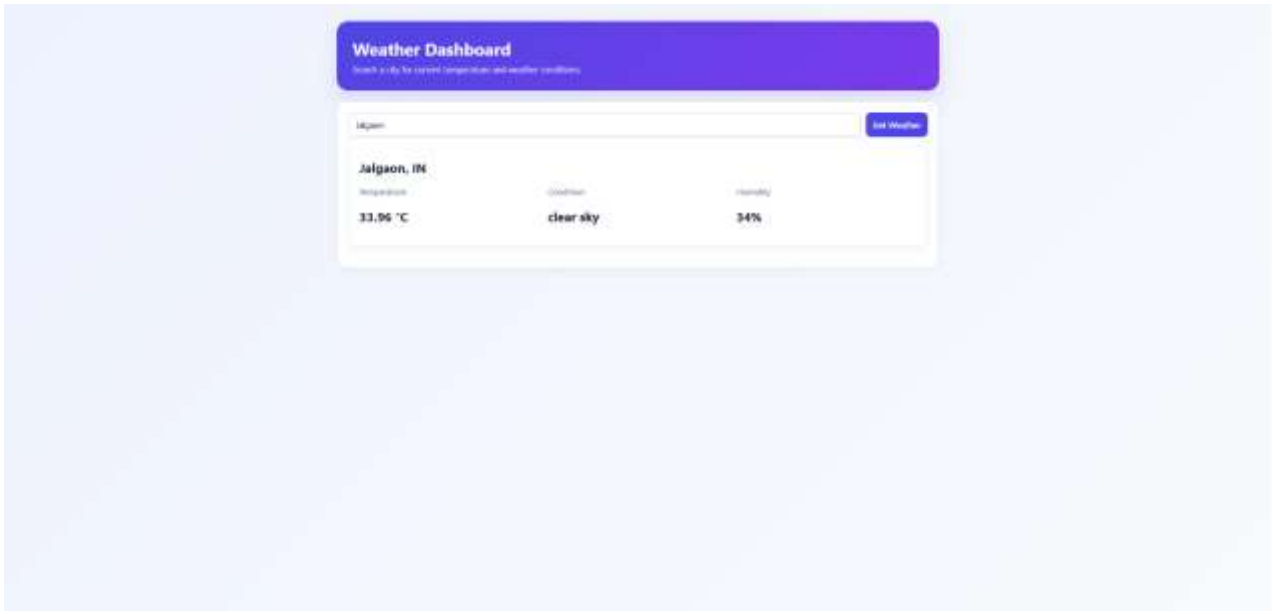
Filename: index.html

```
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1.0"><title>Weather
Dashboard</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container">
<section class="hero"><h1>Weather Dashboard</h1><p>Search a city for current temperature and weather conditions.</p></
section>
<section class="card"><form id="form"><input id="city" placeholder="Enter city, e.g. Pune" required style="flex:
1"><button>Get Weather</button></form><div id="result" class="message">Enter a city to begin.</div></section>
</main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius:  16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
```

```
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```


Weather Dashboard
Jalgaon, IN
33.96 °C
clear sky
34%

Task 10

Question 1

Problem Statement

Create a currency converter that fetches exchange rates asynchronously, validates input, and converts values between currencies.

Code

Filename: app.js

```
// ================================================
// CURRENCY CONVERTER
// ================================================


// ------------------------------------------------------------------------
// DOM HELPER
// ------------------------------------------------------------------------

const $ = (selector) => {
    return document.querySelector(selector);
};


// ------------------------------------------------------------------------
// LIVE API ENDPOINTS
// ------------------------------------------------------------------------
//
// We try Frankfurter first.
//
// If that fails, we try ExchangeRate.host.
//
// If both fail, we use the local fallback rates.
// ------------------------------------------------------------------------

const API_ENDPOINTS = [

    {
        name: "Frankfurter",

        buildUrl: (from, to) => {
            return    `https://api.frankfurter.app/latest?from=${encodeURIComponent(from)}&to=${encodeURIComponent(to)}`;
        }
    },

    {
        name: "ExchangeRate Host",

        buildUrl: (from, to) => {
            return    `https://api.exchangerate.host/latest?base=${encodeURIComponent(from)}&symbols=${encodeURIComponent(
to)}`;
        }
    }

];


// ------------------------------------------------------------------------
// FALLBACK EXCHANGE RATES
// ------------------------------------------------------------------------
//
// These are static demonstration rates.
//
// They are used ONLY when the live API cannot
// be reached.
//
// For a college/lab assignment this prevents the
// application from becoming completely unusable
// when an external API is blocked.
// ------------------------------------------------------------------------

const fallbackRates = {

    USD: {
        EUR: 0.86,
        INR: 83.10,
        GBP: 0.74,
        JPY: 147.50,
        AUD: 1.52,
        CAD: 1.38
    },

    EUR: {
        USD: 1.16,
        INR: 96.60,
```

```
        GBP: 0.86,
        JPY: 171.50,
        AUD: 1.77,
        CAD: 1.60
    },

    INR: {
        USD: 0.0120,
        EUR: 0.0104,
        GBP: 0.0089,
        JPY: 1.77,
        AUD: 0.0183,
        CAD: 0.0166
    },

    GBP: {
        USD: 1.35,
        EUR: 1.16,
        INR: 112.00,
        JPY: 199.00,
        AUD: 2.06,
        CAD: 1.86
    },

    JPY: {
        USD: 0.00678,
        EUR: 0.00583,
        INR: 0.565,
        GBP: 0.00503,
        AUD: 0.0104,
        CAD: 0.00935
    },

    AUD: {
        USD: 0.66,
        EUR: 0.56,
        INR: 54.40,
        GBP: 0.49,
        JPY: 96.90,
        CAD: 0.90
    },

    CAD: {
        USD: 0.72,
        EUR: 0.62,
        INR: 60.20,
        GBP: 0.54,
        JPY: 106.90,
        AUD: 1.11
    }

};


// ------------------------------------------------------------------------------
// SHOW MESSAGE
// ------------------------------------------------------------------------------

function showMessage(text, type = "message") {

    $("#result").className =
        `message ${type}`;

    $("#result").textContent =
        text;
}


// ------------------------------------------------------------------------------
// FETCH JSON
// ------------------------------------------------------------------------------

async function requestJson(url) {

    const controller =
        new AbortController();

    const timeout =
        setTimeout(
            () => controller.abort(),
            10000
        );


    try {

        const response =
            await fetch(
                url,
                {
                    method: "GET",

                    mode: "cors",
```

```
                    cache: "no-store",

                    headers: {
                        "Accept":
                            "application/json"
                    },

                    signal:
                        controller.signal
                }
            );


        if (!response.ok) {

            throw new Error(
                `HTTP ${response.status}`
            );
        }


        const data =
            await response.json();


        return data;

    }

    finally {

        clearTimeout(timeout);
    }
}


// ---------------------------------------------------------------------------
// VALIDATE API RESPONSE
// ---------------------------------------------------------------------------

function normalizeRateResponse(
    data,
    to
) {

    if (
        data &&
        data.rates &&
        data.rates[to] !== undefined
    ) {

        const rate =
            Number(data.rates[to]);


        if (!Number.isFinite(rate)) {

            throw new Error(
                "Invalid exchange rate received."
            );
        }


        return {

            rate: rate,

            date:
                data.date ||
                "Latest available rate"

        };
    }


    throw new Error(
        `No ${to} exchange rate was returned.`
    );
}


// ---------------------------------------------------------------------------
// GET LIVE RATE
// ---------------------------------------------------------------------------

async function fetchLiveRate(
    from,
    to
) {

    let lastError = null;


    for (
```

```
        const endpoint
        of API_ENDPOINTS
    ) {

        try {

            console.log(
                `Trying ${endpoint.name}...`
            );


            const data =
                await requestJson(
                    endpoint.buildUrl(
                        from,
                        to
                    )
                );


            const result =
                normalizeRateResponse(
                    data,
                    to
                );


            return {

                ...result,

                source:
                    endpoint.name

            };

        }

        catch (error) {

            lastError =
                error;


            console.warn(
                `${endpoint.name} failed:`,
                error
            );
        }
    }


    throw lastError ||
        new Error(
            "Live exchange-rate services unavailable."
        );
}


// ---------------------------------------------------------------------------
// GET RATE
// ---------------------------------------------------------------------------
//
// 1. Try live API.
// 2. If API fails, use fallback rate.
// ---------------------------------------------------------------------------

async function getExchangeRate(
    from,
    to
) {

    // Same currency requires no API call.

    if (from === to) {

        return {

            rate: 1,

            source: "Same currency",

            date: "N/A"

        };
    }


    // ---------------------------------------------------------------------
    // TRY LIVE API
    // ---------------------------------------------------------------------

    try {
```

```
            return await fetchLiveRate(
                from,
                to
            );

        }

        catch (error) {

            console.warn(
                "Live exchange-rate services unavailable."
            );

            console.warn(
                "Using local fallback rate.",
                error
            );
        }


        // ------------------------------------------------------------------------
        // FALLBACK
        // ------------------------------------------------------------------------

        const fallback =
            fallbackRates[from]?.[to];


        if (
            fallback !== undefined &&
            Number.isFinite(
                Number(fallback)
            )
        ) {

            return {

                rate:
                    Number(fallback),

                source:
                    "Local fallback rate",

                date:
                    "Static demonstration rate"

            };
        }


        throw new Error(
            `No exchange rate is available for ${from} → ${to}.`
        );
    }


    // ------------------------------------------------------------------------
    // CONVERT CURRENCY
    // ------------------------------------------------------------------------

    async function convert() {

        const amount =
            Number(
                $("#amount").value
            );


        const from =
            $("#from").value;


        const to =
            $("#to").value;


        // ------------------------------------------------------------------------
        // VALIDATE AMOUNT
        // ------------------------------------------------------------------------

        if (
            !Number.isFinite(amount) ||
            amount < 0
        ) {

            showMessage(
                "Please enter a valid non-negative amount.",
                "error"
            );

            return;
        }

        // ------------------------------------------------------------------------
```

```
// SAME CURRENCY
// ------------------------------------------------------------------------

if (from === to) {

    $("#result").className =
        "message success";


    $("#result").innerHTML = `

        <b>
            ${amount.toFixed(2)}
            ${from}
            =
            ${amount.toFixed(2)}
            ${to}
        </b>

        <div class="source-badge">
            Same currency
        </div>

    `;

    return;
}


// ------------------------------------------------------------------------
// LOADING
// ------------------------------------------------------------------------

showMessage(
    "Fetching exchange rate...",
    "warning"
);


try {

    const {

        rate,
        source,
        date

    } = await getExchangeRate(
        from,
        to
    );


    // ------------------------------------------------------------------------
    // VALIDATE RATE
    // ------------------------------------------------------------------------

    if (
        !Number.isFinite(rate) ||
        rate < 0
    ) {

        throw new Error(
            "The exchange-rate service returned an invalid rate."
        );
    }


    // ------------------------------------------------------------------------
    // CALCULATE
    // ------------------------------------------------------------------------

    const converted =
        amount * rate;


    // ------------------------------------------------------------------------
    // DISPLAY RESULT
    // ------------------------------------------------------------------------

    $("#result").className =
        "result-card";


    $("#result").innerHTML = `

        <h2>
            ${amount.toFixed(2)}
            ${from}

            =

            ${converted.toFixed(2)}
            ${to}
```

```
            </h2>


            <p>
                <b>
                    Exchange Rate:
                </b>

                1 ${from}
                =
                ${rate} ${to}
            </p>


            <p class="muted">
                Rate date:
                ${date}
            </p>


            <span class="source-badge">
                Source: ${source}
            </span>

        `;

    }

    catch (error) {

        console.error(
            "Currency conversion failed:",
            error
        );


        showMessage(
            error.message,
            "error"
        );
    }
}


// ------------------------------------------------------------------------
// FORM SUBMISSION
// ------------------------------------------------------------------------

$("#form").addEventListener(
    "submit",
    (event) => {

        event.preventDefault();

        convert();
    }
);
```

Filename: index.html

```
<!doctype html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Currency Converter</title>

    <link rel="stylesheet" href="style.css">
</head>

<body>

    <main class="container">

        <section class="hero">
            <h1>Currency Exchange Converter</h1>

            <p>
                Convert currencies using the latest available exchange rate.
            </p>
        </section>


        <section class="card">

            <form id="form">

                <div class="form-group">
                    <label for="amount">Amount</label>

                    <input
                        id="amount"
```

```
                        type="number"
                        min="0"
                        step="any"
                        value="1"
                        required
                    >
                </div>


                <div class="form-group">
                    <label for="from">From</label>

                    <select id="from">

                        <option value="USD">USD</option>
                        <option value="EUR">EUR</option>
                        <option value="INR">INR</option>
                        <option value="GBP">GBP</option>
                        <option value="JPY">JPY</option>
                        <option value="AUD">AUD</option>
                        <option value="CAD">CAD</option>

                    </select>
                </div>


                <div class="arrow">
                    →
                </div>


                <div class="form-group">
                    <label for="to">To</label>

                    <select id="to">

                        <option value="INR">INR</option>
                        <option value="USD">USD</option>
                        <option value="EUR">EUR</option>
                        <option value="GBP">GBP</option>
                        <option value="JPY">JPY</option>
                        <option value="AUD">AUD</option>
                        <option value="CAD">CAD</option>

                    </select>
                </div>


                <div class="form-group button-group">

                    <label> </label>

                    <button type="submit">
                        Convert
                    </button>

                </div>

            </form>


            <div
                id="result"
                class="message"
            >
                Enter an amount and convert.
            </div>

        </section>

    </main>


    <script src="app.js"></script>

</body>

</html>
```

Filename: style.css

```
:root {
    --bg: #f4f7fb;
    --card: #ffffff;
    --text: #172033;
    --muted: #667085;

    --primary: #4f46e5;
    --primary-dark: #3730a3;

    --danger: #dc2626;
    --success: #059669;
    --warning: #d97706;
```

```
    --border: #e4e7ec;
}


* {
    box-sizing: border-box;
}


body {
    margin: 0;

    min-height: 100vh;

    background:
        linear-gradient(
            135deg,
            #eef2ff,
            #f8fafc
        );

    color: var(--text);

    font-family:
        Inter,
        "Segoe UI",
        Arial,
        sans-serif;
}


.container {
    max-width: 1180px;

    margin: auto;

    padding: 32px 20px;
}


/* ----------------------------------------------------------------------
   HERO
---------------------------------------------------------------------- */

.hero {
    margin-bottom: 20px;

    padding: 30px;

    border-radius: 22px;

    color: #fff;

    background:
        linear-gradient(
            135deg,
            var(--primary),
            #7c3aed
        );

    box-shadow:
        0 18px 40px #4f46e525;
}


.hero h1 {
    margin: 0 0 8px;

    font-size: 32px;
}


.hero p {
    margin: 0;

    opacity: 0.9;
}


/* ----------------------------------------------------------------------
   CARD
---------------------------------------------------------------------- */

.card {
    margin-bottom: 18px;

    padding: 20px;

    border:
        1px solid
        var(--border);

    border-radius: 16px;
```

```
    background: var(--card);

    box-shadow:
        0 8px 25px #10182808;
}


/* ------------------------------------------------------------------------
   FORM
------------------------------------------------------------------------ */

form {
    display: flex;

    align-items: flex-end;

    flex-wrap: wrap;

    gap: 12px;
}


.form-group {
    display: flex;

    flex-direction: column;

    gap: 6px;
}


.form-group label {
    color: var(--muted);

    font-size: 13px;

    font-weight: 600;
}


input,
select,
button {
    padding: 11px 13px;

    border:
        1px solid
        #d0d5dd;

    border-radius: 10px;

    font: inherit;
}


input,
select {
    color: var(--text);

    background: #fff;
}


input:focus,
select:focus {
    outline:
        3px solid
        #4f46e51f;

    border-color:
        var(--primary);
}


button {
    border: 0;

    color: #fff;

    background:
        var(--primary);

    font-weight: 600;

    cursor: pointer;
}


button:hover {
    background:
        var(--primary-dark);
}

button:disabled {
```

```
    opacity: 0.55;

    cursor: not-allowed;
}


.arrow {
    padding: 11px 5px;

    font-size: 24px;

    font-weight: bold;
}


.button-group button {
    min-width: 100px;
}


/* ------------------------------------------------------------------
   RESULT
------------------------------------------------------------------ */

.message {
    margin-top: 18px;

    padding: 15px;

    border-radius: 10px;

     background:
         #f8fafc;
}


.message.error {
    color: var(--danger);

     background:
         #fef3f2;
}


.message.success {
    color: var(--success);

     background:
         #ecfdf3;
}


.message.warning {
    color: var(--warning);

     background:
         #fffaeb;
}


.result-card {
    margin-top: 18px;

    padding: 20px;

    border:
        1px solid
        var(--border);

    border-radius: 14px;

    background: #fff;
}


.result-card h2 {
    margin-top: 0;

    font-size: 26px;
}


.result-card p {
    margin: 8px 0;
}


.muted {
    color: var(--muted);
}


.source-badge {
    display: inline-block;
```

```
    margin-top: 10px;

    padding: 5px 9px;

    border-radius: 99px;

    background: #eef2ff;

    color: #4338ca;

    font-size: 12px;

    font-weight: 700;
}

/* ------------------------------------------------------------------
   RESPONSIVE
------------------------------------------------------------------ */

@media (max-width: 800px) {

    .container {
        padding: 20px 12px;
    }


    .hero h1 {
        font-size: 25px;
    }


    form {
        align-items: stretch;

        flex-direction: column;
    }

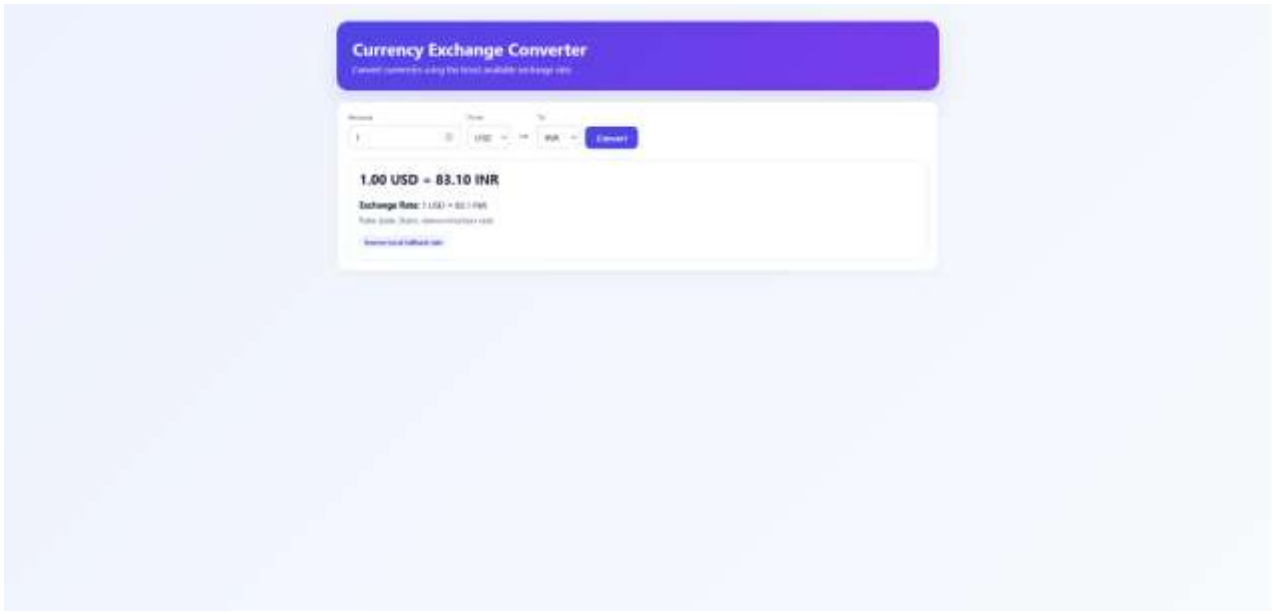

    .form-group,
    .form-group input,
    .form-group select,
    .button-group button {
        width: 100%;
    }


    .arrow {
        text-align: center;

        padding: 0;

        transform: rotate(90deg);
    }

}
```



Task 11

Question 1

Problem Statement

Create a closure-based shopping cart that manages products and cart operations through private closure state.

Code

Filename: app.js

```
function createCart() {
  let items = [];

  function addItem(name, price) {
    if (!name.trim() || !Number.isFinite(price) || price < 0) {
      throw new Error("Invalid item details.");
    }

    items.push({ name: name.trim(), price });
  }

  function removeItem(name) {
    items = items.filter((item) => item.name !== name);
  }

  function getTotalPrice() {
    return items.reduce((total, item) => total + item.price, 0);
  }

  function getItems() {
    return items.map((item) => ({ ...item }));
  }

  return {
    addItem,
    removeItem,
    getTotalPrice,
    getItems
  };
}

const cart = createCart();
const $ = (selector) => document.querySelector(selector);

function render() {
  const items = cart.getItems();

   $("#items").innerHTML = items.length
     ? items.map((item) => `
       <div class="item">
         <b>${item.name}</b>
         <span style="float:right">
           ₹${item.price.toFixed(2)}
           <button class="danger" onclick="removeItem('${item.name}')">Remove</button>
         </span>
       </div>
     `).join("")
     : '<div class="empty">Cart is empty.</div>';

  $("#total").textContent = `₹ ${cart.getTotalPrice().toFixed(2)}`;
}

function removeItem(name) {
  cart.removeItem(name);
  render();
}

$("#form").addEventListener("submit", (event) => {
  event.preventDefault();

  try {
    cart.addItem($("#name").value, Number($("#price").value));
    event.target.reset();
    render();
  } catch (error) {
    alert(error.message);
   }
});

render();
```

Filename: index.html

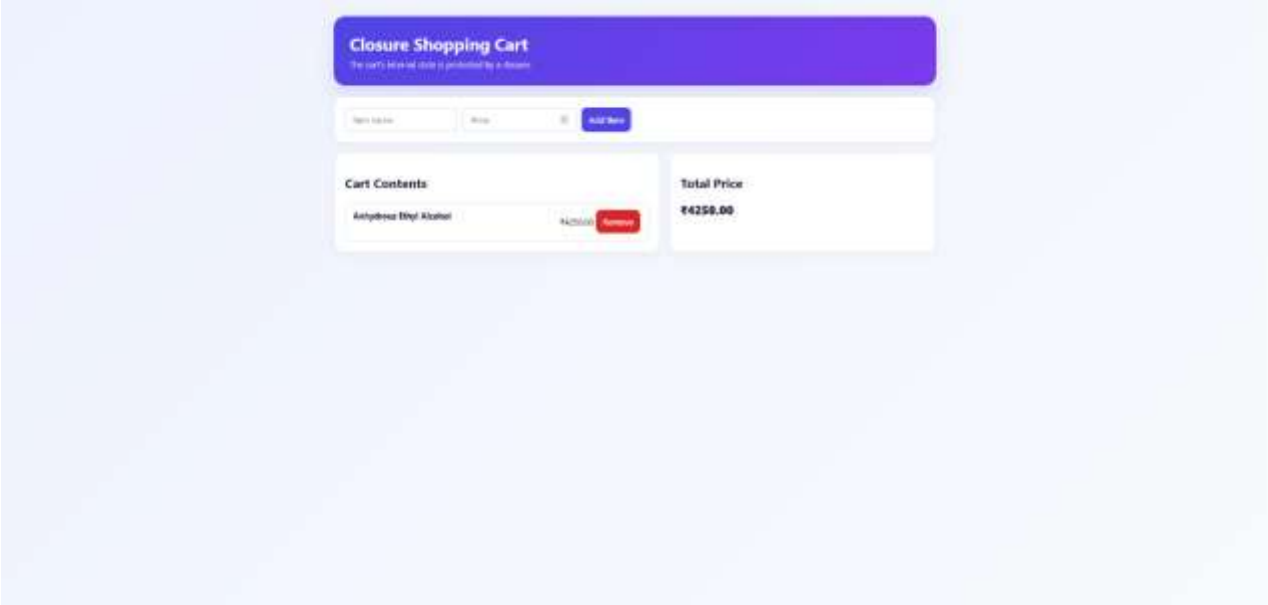
```
<!doctype html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-
scale=1.0"><title>Closure Shopping Cart</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container"><section class="hero"><h1>Closure Shopping Cart</h1><p>The cart's internal state is
```

```
protected by a closure.</p></section><section class="card"><form id="form"><input id="name" placeholder="Item name"
required><input id="price" type="number" min="0" placeholder="Price" required><button>Add Item</button></form></
section><section class="grid2"><div class="card"><h2>Cart Contents</h2><div id="items" class="list"></div></div><div
class="card"><h2>Total Price</h2><div id="total" class="price">₹ 0</div></div></section></main><script src="app.js"></
script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
```

```
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item  {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius:  12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```



Task 12

Question 1

Problem Statement

Create an online shopping cart that supports products, quantities, taxes, discounts, totals, and closure-based state management.

Code

Filename: app.js

```
const products = [
  { id: "P001", name: "Laptop", price: 65000 },
  { id: "P002", name: "Headphones", price: 3500 },
  { id: "P003", name: "Keyboard", price: 1800 },
  { id: "P004", name: "Mouse", price: 900 }
];

function createSecureCart() {
  let cart = [];

  function addProduct(product) {
    const existing = cart.find((item) => item.id === product.id);

    if (existing) {
      existing.quantity += 1;
    } else {
      cart.push({ ...product, quantity: 1 });
    }
  }

  function updateQuantity(id, quantity) {
    const item = cart.find((product) => product.id === id);

    if (!item) return;

    if (quantity <= 0) {
      removeProduct(id);
      return;
    }

    item.quantity = quantity;
  }

  function removeProduct(id) {
    cart = cart.filter((item) => item.id !== id);
  }

  function getSnapshot() {
    return cart.map((item) => ({ ...item }));
  }

  function getBill() {
    const subtotal = cart.reduce(
      (sum, item) => sum + item.price * item.quantity,
      0
    );

    const discount = subtotal >= 5000 ? subtotal * 0.10 : 0;
    const taxableAmount = subtotal - discount;
    const tax = taxableAmount * 0.18;

    return {
      subtotal,
      discount,
      tax,
      total: taxableAmount + tax
    };
  }

  return Object.freeze({
    addProduct,
    updateQuantity,
    removeProduct,
    getSnapshot,
    getBill
  });
}

const cart = createSecureCart();
const $ = (selector) => document.querySelector(selector);

$("#products").innerHTML = products.map((product) => `
  <div class="product">
    <h3>${product.name}</h3>
    <div class="price">₹ ${product.price.toLocaleString()}</div>
```

```
    <button onclick="addProduct('${product.id}')">Add to Cart</button>
  </div>
`).join("");

function addProduct(id) {
  const product = products.find((item) => item.id === id);

  if (!product) return;

  cart.addProduct(product);
  render();
}

function updateQuantity(id, quantity) {
  cart.updateQuantity(id, Number(quantity));
  render();
}

function removeProduct(id) {
  cart.removeProduct(id);
  render();
}

function render() {
  const items = cart.getSnapshot();

   $("#cart").innerHTML = items.length
     ? items.map((item) => `
       <div class="item">
         <b>${item.name}</b>
         <p>₹ ${item.price.toFixed(2)} ×
           <input
             type="number"
             min="1"
             value="${item.quantity}"
             style="width:75px"
             onchange="updateQuantity('${item.id}',  this.value)"
           >
         </p>
          <button class="danger" onclick="removeProduct('${item.id}')">Remove</button>
       </div>
    `).join("")
    : '<div class="empty">Cart is empty.</div>';

  const bill = cart.getBill();

  $("#bill").innerHTML = `
    <p>Subtotal: ₹ ${bill.subtotal.toFixed(2)}</p>
    <p>Discount: −₹ ${bill.discount.toFixed(2)}</p>
    <p>Tax (18%): ₹ ${bill.tax.toFixed(2)}</p>
    <hr>
    <h2>Total: ₹ ${bill.total.toFixed(2)}</h2>
  `;
}

render();
```

Filename: index.html

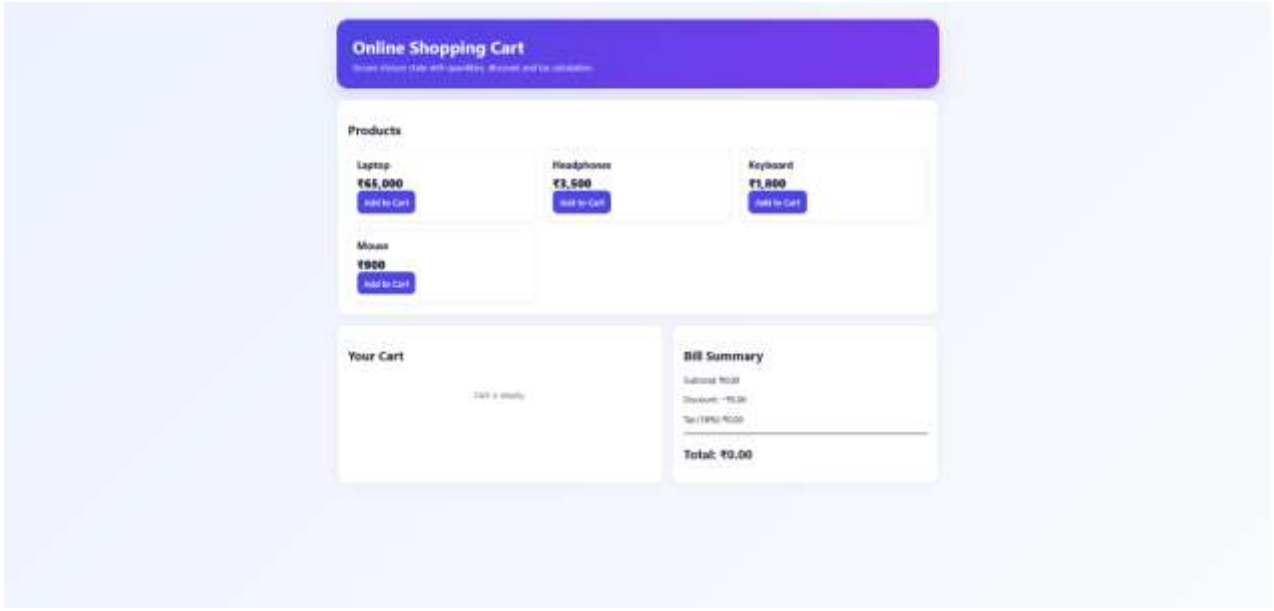
```
<!doctype html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-
scale=1.0"><title>Online Shopping Cart</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container"><section class="hero"><h1>Online Shopping Cart</h1><p>Secure closure state with
quantities, discount and tax calculation.</p></section><section class="card"><h2>Products</h2><div id="products"
class="product-grid"></div></section><section class="grid2"><div class="card"><h2>Your Cart</h2><div id="cart"
class="list"></div></div><div class="card"><h2>Bill Summary</h2><div id="bill"></div></div></section></main><script
src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
```

```
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
```

```
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```



Task 13

Question 1

Problem Statement

Create a student result analysis application that uses higher-order functions to calculate and display result statistics.

Code

Filename: app.js

```
const students = [
  {
    id: "S001",
    name: "Aarav",
    marks: { Maths: 88, Science: 92, English: 81, JavaScript: 95 }
  },
  {
    id: "S002",
    name: "Diya",
    marks: { Maths: 76, Science: 84, English: 79, JavaScript: 88 }
  },
  {
    id: "S003",
    name: "Kabir",
    marks: { Maths: 42, Science: 55, English: 38, JavaScript: 61 }
  },
  {
    id: "S004",
    name: "Meera",
    marks: { Maths: 95, Science: 91, English: 94, JavaScript: 98 }
  },
  {
    id: "S005",
    name: "Riya",
    marks: { Maths: 31, Science: 44, English: 36, JavaScript: 49 }
  }
];

const $ = (selector) => document.querySelector(selector);

function calculatePercentage(student) {
  const marks = Object.values(student.marks);

  return marks.reduce((sum, mark) => sum + mark, 0) / marks.length;
}

function getGrade(percentage) {
  if (percentage >= 90) return "A+";
  if (percentage >= 80) return "A";
  if (percentage >= 70) return "B";
  if (percentage >= 60) return "C";
  if (percentage >= 50) return "D";
  if (percentage >= 40) return "E";
  return "F";
}

const analyzedStudents = students.map((student) => {
  const percentage = calculatePercentage(student);

  return {
    ...student,
    percentage,
    grade: getGrade(percentage),
    passed: Object.values(student.marks).every((mark) => mark >= 40)
  };
});

const classAverage =
  analyzedStudents.reduce((sum, student) => sum + student.percentage, 0) /
  analyzedStudents.length;

const passPercentage =
  analyzedStudents.filter((student) => student.passed).length /
  analyzedStudents.length *
  100;

const subjectToppers = Object.keys(students[0].marks).map((subject) => {
  const topper = students.reduce((best, student) =>
    student.marks[subject] > best.marks[subject] ? student : best
  );

  return { subject, student: topper };
});

function render() {
```

```
  const query = $("#search").value.trim().toLowerCase();

  const filtered = analyzedStudents
    .filter((student) =>
      student.id.toLowerCase().includes(query) ||
      student.name.toLowerCase().includes(query)
    )
    .sort((a, b) => b.percentage - a.percentage);

  $("#rows").innerHTML = filtered.length
    ? filtered.map((student) => `
      <tr>
        <td>${student.id}</td>
        <td>${student.name}</td>
        <td>${student.percentage.toFixed(2)}%</td>
        <td>${student.grade}</td>
        <td class="${student.passed ? "success-text" : "danger-text"}">
          ${student.passed ? "Pass" : "Fail"}
        </td>
      </tr>
    `).join("")
    : '<tr><td colspan="5" class="empty">No student found.</td></tr>';

  $("#average").textContent = `${classAverage.toFixed(2)}%`;
  $("#pass").textContent = `${passPercentage.toFixed(2)}%`;

  const topper = [...analyzedStudents].sort(
    (a, b) => b.percentage - a.percentage
  )[0];

  $("#topper").textContent = `${topper.name} (${topper.percentage.toFixed(2)}%)`;

  $("#subjectToppers").innerHTML = subjectToppers.map((item) =>
    `<span class="badge">${item.subject}: ${item.student.name}</span>`
  ).join("");
}

$("#search").addEventListener("input", render);

render();
```

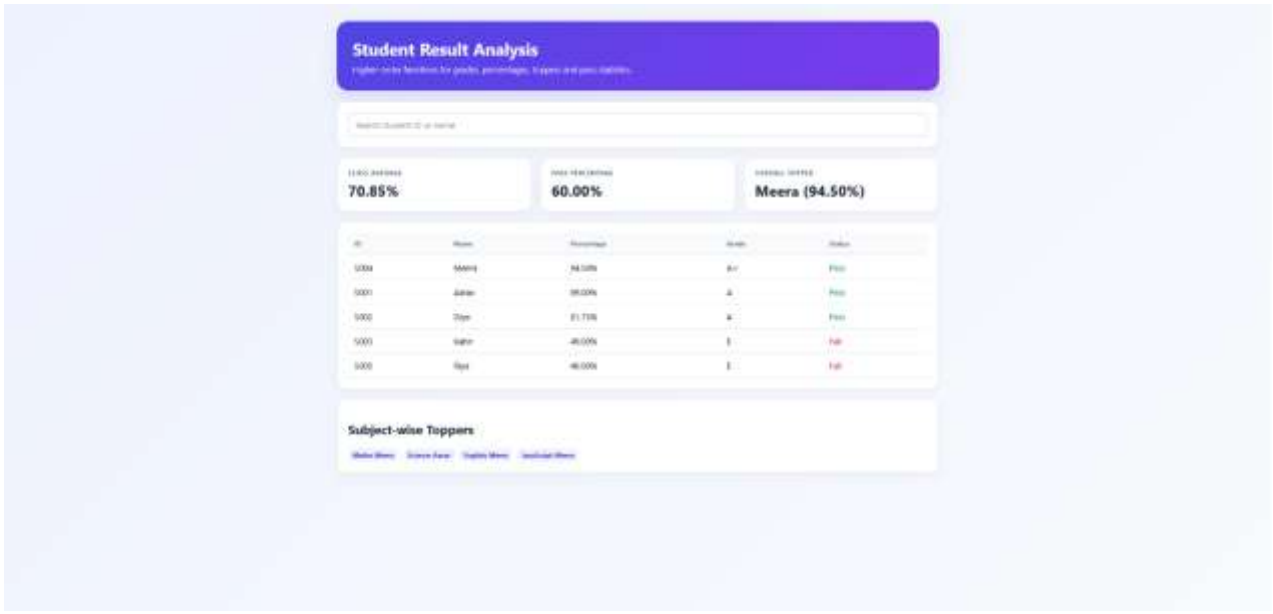
Filename: index.html

```
<!doctype html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-
scale=1.0"><title>Student Result Analysis</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container"><section class="hero"><h1>Student Result Analysis</h1><p>Higher-order functions for
grades, percentages, toppers and pass statistics.</p></section><section class="card"><input id="search"
placeholder="Search student ID or name" style="width:100%"></section><section class="grid"><div class="card
stat"><b>Class Average</b><strong id="average">0%</strong></div><div class="card stat"><b>Pass Percentage</b><strong
id="pass">0%</strong></div><div class="card stat"><b>Overall Topper</b><strong id="topper">—</strong></div></
section><section class="card"><table><thead><tr><th>ID</th><th>Name</th><th>Percentage</th><th>Grade</th><th>Status</
th></tr></thead><tbody id="rows"></tbody></table></section><section class="card"><h2>Subject-wise Toppers</h2><div
id="subjectToppers" class="pillbar"></div></section></main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius:  16px;
  background: var(--card);
```

```
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
}
```


Student Result Analysis
70.85%
60.00%
Meera (94.50%)
Subject-wise Toppers

Task 14

Question 1

Problem Statement

Create an online banking application with account creation, deposits, withdrawals, transfers, balances, transaction history, persistence, callbacks, promises, and async/await.

Code

Filename: app.js

```
const STORAGE_KEY = "online_banking_corrected";

class Bank {
  #accounts;
  #nextId;

  constructor() {
    const saved = JSON.parse(localStorage.getItem(STORAGE_KEY) || "null");

    this.#accounts = new Map(
      saved?.accounts?.map((account) => [account.id, account]) || []
    );

    this.#nextId = saved?.nextId || 1001;
  }

  #save() {
    localStorage.setItem(
      STORAGE_KEY,
      JSON.stringify({
        accounts: [...this.#accounts.values()],
        nextId: this.#nextId
      })
    );
  }

  #getAccount(id) {
    const account = this.#accounts.get(String(id));

    if (!account) {
      throw new Error("Account not found.");
    }

    return account;
  }

  #simulateServer(result) {
    return new Promise((resolve) => {
      setTimeout(() => resolve(result), 500);
    });
  }

  createAccount(name, openingBalance, callback) {
    try {
      const cleanName = name.trim();

      if (!cleanName) {
        throw new Error("Account holder name is required.");
      }

      if (!Number.isFinite(openingBalance) || openingBalance < 0) {
        throw new Error("Opening balance must be non-negative.");
      }

      const id = `AC${this.#nextId++}`;

      const account = {
        id,
        name: cleanName,
        balance: openingBalance,
        history: [
          {
            type: "ACCOUNT CREATED",
            amount: openingBalance,
            timestamp: new Date().toLocaleString()
          }
        ]
      };

      this.#accounts.set(id, account);
      this.#save();

      // Callback demonstrates the requested callback concept.
      callback(null, this.getPublicAccount(account));
    } catch (error) {
```

```
    callback(error);
  }
}

getPublicAccount(account) {
  return {
    id: account.id,
    name: account.name,
    balance: account.balance,
    history: account.history.map((item) => ({ ...item }))
  };
}

getAccounts() {
  return [...this.#accounts.values()].map((account) =>
    this.getPublicAccount(account)
  );
}

async deposit(id, amount) {
  const account = this.#getAccount(id);

  if (!Number.isFinite(amount) || amount <= 0) {
    throw new Error("Deposit amount must be greater than zero.");
  }

  account.balance += amount;
  account.history.push({
    type: "DEPOSIT",
    amount,
    timestamp: new Date().toLocaleString()
  });

  this.#save();

  return this.#simulateServer(this.getPublicAccount(account));
}

async withdraw(id, amount) {
  const account = this.#getAccount(id);

  if (!Number.isFinite(amount) || amount <= 0) {
    throw new Error("Withdrawal amount must be greater than zero.");
  }

  if (amount > account.balance) {
    throw new Error("Insufficient account balance.");
  }

  account.balance -= amount;
  account.history.push({
    type: "WITHDRAWAL",
    amount,
    timestamp: new Date().toLocaleString()
  });

  this.#save();

  return this.#simulateServer(this.getPublicAccount(account));
}

async transfer(fromId, toId, amount) {
  const sender = this.#getAccount(fromId);
  const receiver = this.#getAccount(toId);

  if (sender.id === receiver.id) {
    throw new Error("Source and destination accounts must be different.");
  }

  if (!Number.isFinite(amount) || amount <= 0) {
    throw new Error("Transfer amount must be greater than zero.");
  }

  if (amount > sender.balance) {
    throw new Error("Insufficient account balance.");
  }

  sender.balance -= amount;
  receiver.balance += amount;

  const timestamp = new Date().toLocaleString();

  sender.history.push({
    type: "TRANSFER OUT",
    amount,
    to: receiver.id,
    timestamp
  });

  receiver.history.push({
    type: "TRANSFER IN",
    amount,
    from: sender.id,
    timestamp
```

```
    });

    this.#save();

    return this.#simulateServer({
      sender: this.getPublicAccount(sender),
      receiver: this.getPublicAccount(receiver)
    });
  }

  async getBalance(id) {
    const account = this.#getAccount(id);
    return this.#simulateServer(this.getPublicAccount(account));
  }
}

const bank = new Bank();

const $ = (selector) => document.querySelector(selector);

function showMessage(selector, text, type = "success") {
  $(selector).innerHTML = `<div class="message ${type}">${text}</div>`;
}

function renderAccounts() {
  const accounts = bank.getAccounts();

  $("#accounts").innerHTML = accounts.length
    ? accounts.map((account) => `
      <div class="item">
        <h3>${account.name}</h3>
        <p><b>Account ID:</b> ${account.id}</p>
        <p><b>Balance:</b> ₹ ${account.balance.toFixed(2)}</p>
      </div>
    `).join("")
    : '<div class="empty">No accounts created yet.</div>';

  const currentSource = $("#sourceAccount").value;
  const currentDestination = $("#destinationAccount").value;

  const options = accounts.map((account) =>
    `<option value="${account.id}">${account.id} — ${account.name}</option>`
  ).join("");

  $("#sourceAccount").innerHTML =
    accounts.length
      ? `<option value="">Select source account</option>${options}`
      : `<option value="">No accounts available</option>`;

  $("#destinationAccount").innerHTML =
    accounts.length
      ? `<option value="">Select destination</option>${options}`
      : `<option value="">No accounts available</option>`;

  if (accounts.some((account) => account.id === currentSource)) {
    $("#sourceAccount").value = currentSource;
  }

  if (accounts.some((account) => account.id === currentDestination)) {
    $("#destinationAccount").value = currentDestination;
  }

  renderDetails();
}

function renderDetails() {
  const id = $("#sourceAccount").value;

  if (!id) {
    $("#details").innerHTML = '<div class="empty">Select an account to view details.</div>';
    return;
  }

  const account = bank.getAccounts().find((item) => item.id === id);

  if (!account) {
    $("#details").innerHTML = '<div class="empty">Account not found.</div>';
    return;
  }

  $("#details").innerHTML = `
    <div class="grid">
      <div>
        <span class="muted">Account Holder</span>
        <h2>${account.name}</h2>
      </div>
      <div>
        <span class="muted">Account ID</span>
        <h2>${account.id}</h2>
      </div>
      <div>
        <span class="muted">Balance</span>
        <h2>₹ ${account.balance.toFixed(2)}</h2>
      </div>
```

```
    </div>
    <h3>Transaction History</h3>
    <table>
      <thead>
        <tr><th>Type</th><th>Amount</th><th>Details</th><th>Time</th></tr>
      </thead>
      <tbody>
        ${account.history.map((transaction) => `
          <tr>
            <td>${transaction.type}</td>
            <td>₹ ${Number(transaction.amount).toFixed(2)}</td>
            <td>${transaction.to || transaction.from || "—"}</td>
            <td>${transaction.timestamp}</td>
          </tr>
        `).join("")}
      </tbody>
    </table>
  `;
}

$("#createForm").addEventListener("submit", (event) => {
  event.preventDefault();

  const name = $("#accountName").value;
  const openingBalance = Number($("#openingBalance").value);

  bank.createAccount(name, openingBalance, (error, account) => {
    if (error) {
      showMessage("#createMessage", error.message, "error");
      return;
    }

    showMessage(
      "#createMessage",
      `Account created successfully. ID: <b>${account.id}</b>, Holder: <b>${account.name}</b>, Opening Balance: <b>₹ ${
account.balance.toFixed(2)}</b>.`
    );

    $("#createForm").reset();
    $("#openingBalance").value = 0;

    renderAccounts();

    // Automatically select the newly created account.
    $("#sourceAccount").value = account.id;
    renderDetails();
  });
});

async function perform(operation) {
  const sourceId = $("#sourceAccount").value;
  const amount = Number($("#amount").value);

  if (!sourceId) {
    showMessage("#transactionMessage", "Select a source account first.", "error");
    return;
  }

  try {
    let result;

    if (operation === "deposit") {
      result = await bank.deposit(sourceId, amount);
      showMessage(
        "#transactionMessage",
        `Deposit successful. New balance: <b>₹ ${result.balance.toFixed(2)}</b>.`
      );
    }

    if (operation === "withdraw") {
      result = await bank.withdraw(sourceId, amount);
      showMessage(
        "#transactionMessage",
        `Withdrawal successful. New balance: <b>₹ ${result.balance.toFixed(2)}</b>.`
      );
    }

    if (operation === "transfer") {
      const destinationId = $("#destinationAccount").value;

      if (!destinationId) {
        showMessage("#transactionMessage", "Select a destination account.", "error");
        return;
      }

      result = await bank.transfer(sourceId, destinationId, amount);

      showMessage(
        "#transactionMessage",
        `Transfer successful. Sender balance: <b>₹ ${result.sender.balance.toFixed(2)}</b>.`
      );
    }

    $("#amount").value = "";
```

```
    renderAccounts();
    $("#sourceAccount").value = sourceId;
    renderDetails();
  } catch (error) {
    showMessage("#transactionMessage", error.message, "error");
  }
}

$("#depositButton").addEventListener("click", () => perform("deposit"));
$("#withdrawButton").addEventListener("click", () => perform("withdraw"));
$("#transferButton").addEventListener("click", () => perform("transfer"));

$("#balanceButton").addEventListener("click", async () => {
  const sourceId = $("#sourceAccount").value;

  if (!sourceId) {
    showMessage("#transactionMessage", "Select an account first.", "error");
    return;
  }

  try {
    const account = await bank.getBalance(sourceId);

    showMessage(
      "#transactionMessage",
      `Current balance of ${account.name} (${account.id}): <b>₹${account.balance.toFixed(2)}</b>.`
    );

     renderDetails();
   } catch (error) {
    showMessage("#transactionMessage", error.message, "error");
   }
});

$("#sourceAccount").addEventListener("change", renderDetails);

renderAccounts();
```

Filename: index.html

```
<!doctype html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1.0"><title>Online Banking
Simulator</title><link rel="stylesheet" href="style.css"></head>
<body><main class="container">
<section class="hero"><h1>Online Banking Transaction Simulator</h1><p>Callbacks, Promises, async/await and closures in
a functional banking UI.</p></section>
<section class="grid2">
  <div class="card">
    <h2>Create Account</h2>
    <form id="createForm">
      <input id="accountName" placeholder="Account holder name" required>
      <input id="openingBalance" type="number" min="0" step=".01" placeholder="Opening balance" value="0">
      <button>Create Account</button>
    </form>
    <div id="createMessage"></div>
    <div id="accounts" class="list" style="margin-top:14px"></div>
  </div>

  <div class="card">
    <h2>Transactions</h2>
    <label>Source Account</label>
    <select id="sourceAccount" style="width:100%;margin:6px 0 12px"></select>

    <label>Amount</label>
    <input id="amount" type="number" min=".01" step=".01" placeholder="Amount" style="width:100%;margin:6px 0 12px">

    <label>Destination Account — Transfer only</label>
    <select id="destinationAccount" style="width:100%;margin:6px 0 12px"></select>

    <div class="actions">
      <button id="depositButton">Deposit</button>
      <button id="withdrawButton">Withdraw</button>
      <button id="transferButton">Transfer</button>
      <button id="balanceButton" class="secondary">Balance</button>
    </div>
    <div id="transactionMessage"></div>
  </div>
</section>

<section class="card">
  <h2>Selected Account</h2>
  <div id="details" class="empty">Create an account to begin.</div>
</section>
</main><script src="app.js"></script></body></html>
```

Filename: style.css

```
:root {
  --bg: #f4f7fb;
  --card: #ffffff;
  --text: #172033;
  --muted: #667085;
```

```
  --primary: #4f46e5;
  --primary-dark: #3730a3;
  --danger: #dc2626;
  --success: #059669;
  --warning: #d97706;
  --border: #e4e7ec;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef2ff, #f8fafc);
  color: var(--text);
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}
.container { max-width: 1180px; margin: auto; padding: 32px 20px; }
.hero {
  margin-bottom: 20px;
  padding: 30px;
  border-radius: 22px;
  color: #fff;
  background: linear-gradient(135deg, var(--primary), #7c3aed);
  box-shadow: 0 18px 40px #4f46e525;
}
.hero h1 { margin: 0 0 8px; font-size: 32px; }
.hero p { margin: 0; opacity: .9; }
.card {
  margin-bottom: 18px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: var(--card);
  box-shadow: 0 8px 25px #10182808;
}
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
.grid2 { display: grid; grid-template-columns: 1.1fr .9fr; gap: 18px; }
.stat b {
  display: block;
  color: var(--muted);
  font-size: 12px;
  letter-spacing: .06em;
  text-transform: uppercase;
}
.stat strong { display: block; margin-top: 8px; font-size: 28px; }
form, .controls, .actions { display: flex; flex-wrap: wrap; gap: 10px; }
input, select, button, textarea {
  padding: 11px 13px;
  border: 1px solid #d0d5dd;
  border-radius: 10px;
  font: inherit;
}
input, select, textarea { color: var(--text); background: #fff; }
input:focus, select:focus, textarea:focus {
  outline: 3px solid #4f46e51f;
  border-color: var(--primary);
}
button {
  border: 0;
  color: #fff;
  background: var(--primary);
  font-weight: 600;
  cursor: pointer;
}
button:hover { background: var(--primary-dark); }
button.secondary { background: #475467; }
button.danger { background: var(--danger); }
button.success { background: var(--success); }
button.warning { background: var(--warning); }
button:disabled { opacity: .55; cursor: not-allowed; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid var(--border); text-align: left; }
th { color: var(--muted); background: #f8fafc; font-size: 13px; }
.badge {
  display: inline-block;
  padding: 5px 9px;
  border-radius: 99px;
  background: #eef2ff;
  color: #4338ca;
  font-size: 12px;
  font-weight: 700;
}
.success-text { color: var(--success); }
.danger-text { color: var(--danger); }
.warning-text { color: var(--warning); }
.muted { color: var(--muted); }
.message { margin-top: 12px; padding: 12px; border-radius: 10px; background: #f8fafc; }
.message.error { color: var(--danger); background: #fef3f2; }
.message.success { color: var(--success); background: #ecfdf3; }
.message.warning { color: var(--warning); background: #fffaeb; }
.empty { padding: 28px; text-align: center; color: var(--muted); }
.footer { padding: 14px; color: var(--muted); text-align: center; font-size: 13px; }
.item {
  padding: 14px;
  border: 1px solid var(--border);
```

```
  border-radius: 12px;
  background: #fff;
}
.list { display: grid; gap: 10px; }
.movie-grid, .product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}
.movie, .product {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
}
.movie h3, .product h3 { margin: 0 0 8px; }
.price { font-size: 22px; font-weight: 800; }
.pillbar { display: flex; flex-wrap: wrap; gap: 8px; }
.seat-grid { display: grid; grid-template-columns: repeat(8, 1fr); gap: 8px; }
.seat { color: var(--text); background: #e4e7ec; }
.seat.selected { color: #fff; background: var(--primary); }
.seat.booked { color: #fff; background: #98a2b3; }
@media (max-width: 800px) {
  .grid, .grid2, .movie-grid, .product-grid { grid-template-columns: 1fr; }
  .seat-grid { grid-template-columns: repeat(4, 1fr); }
  .container { padding: 20px 12px; }
  .hero h1 { font-size: 25px; }
```

